

Chapter 2

Application

~~Layer~~

A note on the

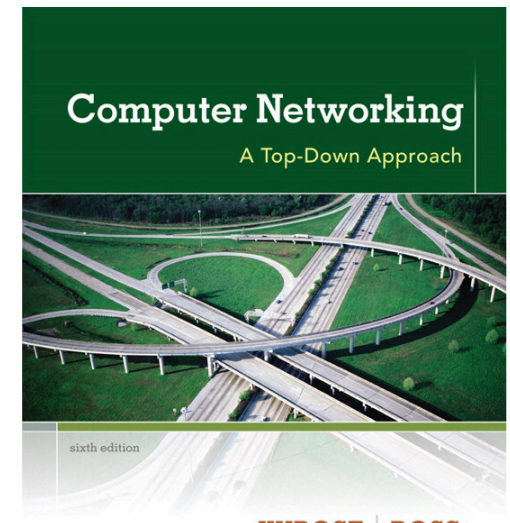
We're making these

They're in PowerPoint form so you see the animations; and can add, modify, and delete slides (including this one) and slide content to suit your needs. They obviously represent a *lot* of work on our part. In return for use, we only ask the following:

- ❖ If you use these slides (e.g., in a class) that you mention their source (after all, we'd like people to use our book!)
- ❖ If you post any slides on a www site, that you note that they are adapted from (or perhaps identical to) our slides, and note our copyright of this material.

Thanks and enjoy! JFK/KWR

©All material copyright 1996-2012
J.F Kurose and K.W. Ross, All Rights Reserved



*Networking: A
Top Down
Approach
6th edition
Jim Kurose, Keith
Ross
Addison-Wesley
March 2012*

Chapter 2: outline

2.1 principles of
network
applications

2.6 P2P applications
2.7 socket
gramming

2.4 electronic mail

- SMTP, POP3,
IMAP

2.5 DNS

Chapter 2: application layer

our goals:

- ❖ conceptual, implementation aspects of network
- ❖ learn about protocols by examining popular application-level
 - SMTP / POP3 / IMAP
 - DNS
- ❖ creating network applications
 - socket API
- transport service models
- client-server paradigm
- peer-to-peer paradigm

Some network apps

- ❖ e-mail
- ❖ web
- ❖ text messaging
- ❖ voice over IP (e.g., Skype)
- ❖ time video encoding
- ❖ multi-games
- ❖ ...
- ❖ ...
- ❖ streaming stored video (YouTube, Hulu, Netflix)

Creating a network app

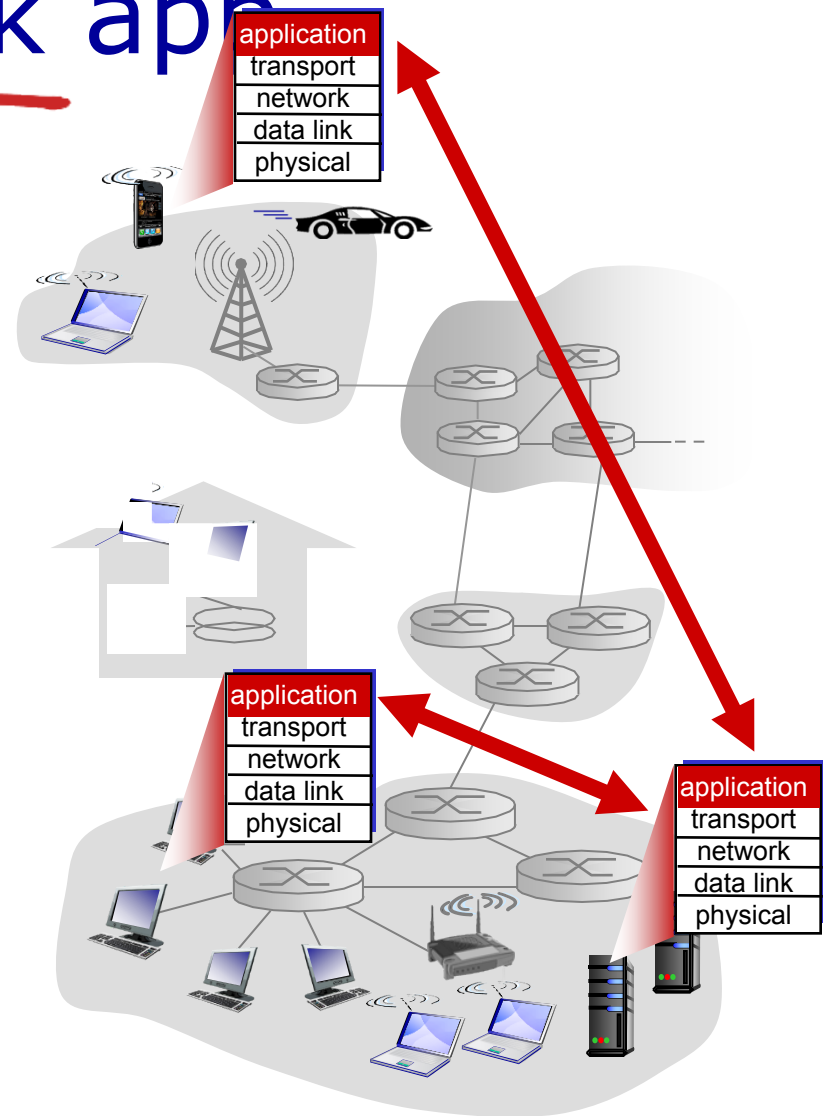
write programs that:

- ❖ run on (different) *end systems*
- ❖ communicate over network

with b

no need to write software for network-core devices

- ❖ network-core devices do not run user applications
- ❖ applications on end systems allows for

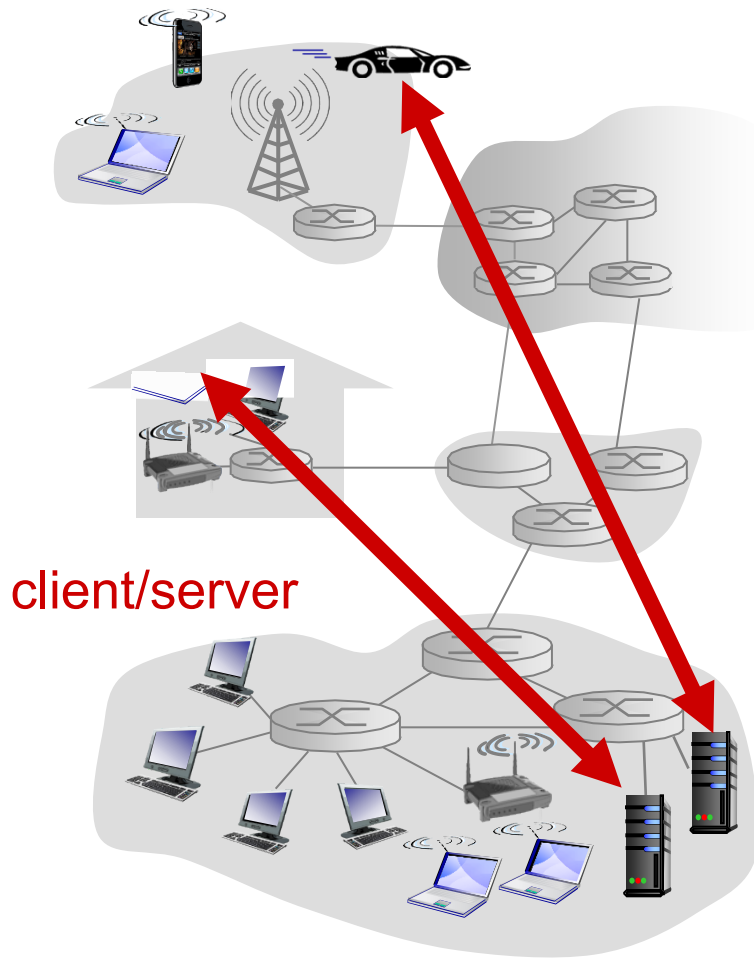


Application architectures

possible structure of applications:

- ❖ client-server
- ❖ peer-to-peer

Client-server architecture

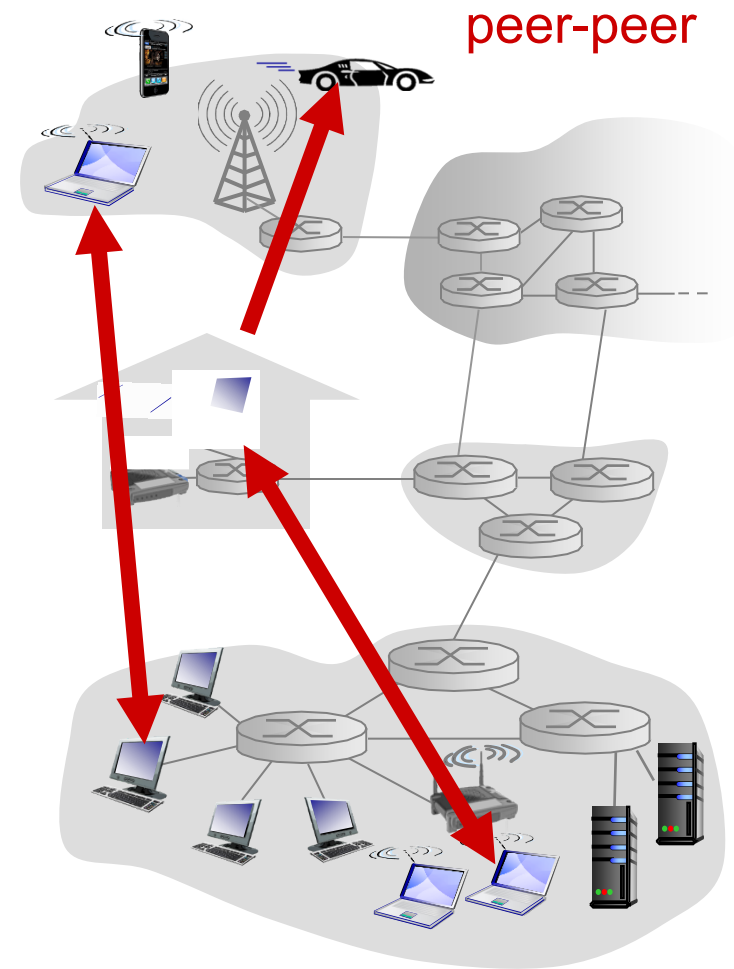


server:

- ❖ always-on host
- ❖ permanent IP address
centers for scaling
- ❖ communicate with server
- ❖ may be intermittently connected
- ❖ may have dynamic IP addresses
- ❖ do not communicate directly with each other

P2P architecture

- ❖ no always-on server
- ❖ arbitrary end systems directly communicate
- ❖ peers request service
- *self scalability* – new peers bring new service capacity, as well as new service demands
- ❖ peers are intermittently connected and change IP addresses



Processes communicating

process: program running within a host

comm

inter-communication

(defined by OS)

- ❖ processes in different hosts communicate by exchanging *messages*

clients, servers

client process:
process that initiates

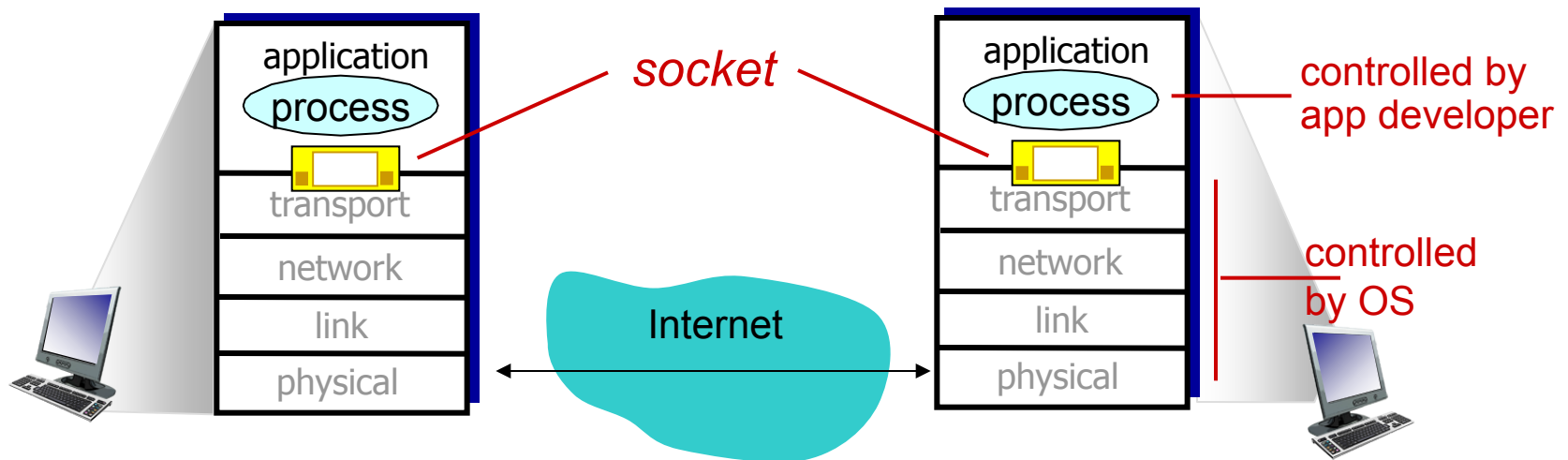
waits to

be contacted

- ❖ aside: applications with P2P architectures have client processes & server processes

Sockets

- ❖ process sends/receives messages to/from its **socket**
- ❖ socket analogous to door
 - sending process shoves message out door



Addressing processes

- ❖ to receive messages, process must have *identifier*
- ❖ host device has
 - ❖ *identifier* includes both IP address and port numbers associated with process on host.
- ❖ Q: do host on which process runs A: no, not really
 - identifying the process can be done by looking at the process running on same host
- ❖ to send HTTP message to gaia.cs.umass.edu web server:
 - mail server: 25
 - IP address: 128.119.245.12
 - port number: 80
- ❖ more shortly...

App-layer protocol defines

- ❖ types of messages exchanged,

- e.g., request, response

message fields are delineated

- ❖ message semantics

- meaning of information in fields

- ❖ rules for when and how processes send & respond to messages

- open protocols:

- ❖ defined in RFCs
- ❖ allows for interoperability



What transport service does an app need?

data integrity

- ❖ some apps (e.g., file transfer, web transactions) require 100% reliable d

- ❖ other a
can tolerate some loss

- ❖ some apps (e.g., Internet telephony, interactive games) require low delay to be “effective”

throughput

- ❖ some apps (e.g., multimedia) require minimum amount of oughput to be

use of
whatever throughput
they get

security

- ❖ encryption, data integrity, ...

Transport service requirements:

~~common apps~~

application	data loss	throughput	time sensitive
file transfer	no		no
real-time audio			s, 100's msec
stored audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	few kbps up	yes, 100's msec
text messaging	no loss	elastic	yes and no

Internet transport protocols ~~services~~

TCP service:

- ❖ *reliable transport* between sending and receiving process
- ❖ *congestion control:* throttle sender when network overloaded
- ❖ *does not provide:* timing, minimum throughput guarantee, security
- ❖ *connection-oriented:* setup required between client and server processes

UDP service:

- ❖ *unreliable data transfer* between sending and

control, congestion control, timing, throughput guarantee, security, or connection setup,

Q: why bother? Why is there a UDP?

Internet apps: application, transport protocols

	application	application layer protocol	underlying transport protocol
rem	e-mail		TCP
streaming m	fil		TCP or UDP
Internet telephony		RTP [RFC 1889] SIP, RTP, proprietary (e.g., Skype)	TCP or UDP

Securing TCP

TCP & UDP

- ❖ no encryption
- ❖ cleartext pas

in cle

SSL

- ❖ provides encrypted TCP connection
- ❖ data integrity
- ❖ end-point authentication

SSL is at app layer

- ❖ Apps use SSL
ies, which
- ❖ cleartext passwds sent into socket traverse Internet encrypted
- ❖ See Chapter 7

Chapter 2: outline

2.1 principles of
network
applications

2.6 P2P applications
2.7 socket
gramming

2.2 We

2.3 FTP

2.4 electronic mail

- SMTP, POP3,
IMAP

2.5 DNS

Web and HTTP

First, a review...

- ❖ *web page* consists of *objects*
- ❖ object can be EG image,

whic
objects

- ❖ each object is addressable by a *URL*,
e.g., `www.someschool.edu/someDept/pic.gif`

host name

path name

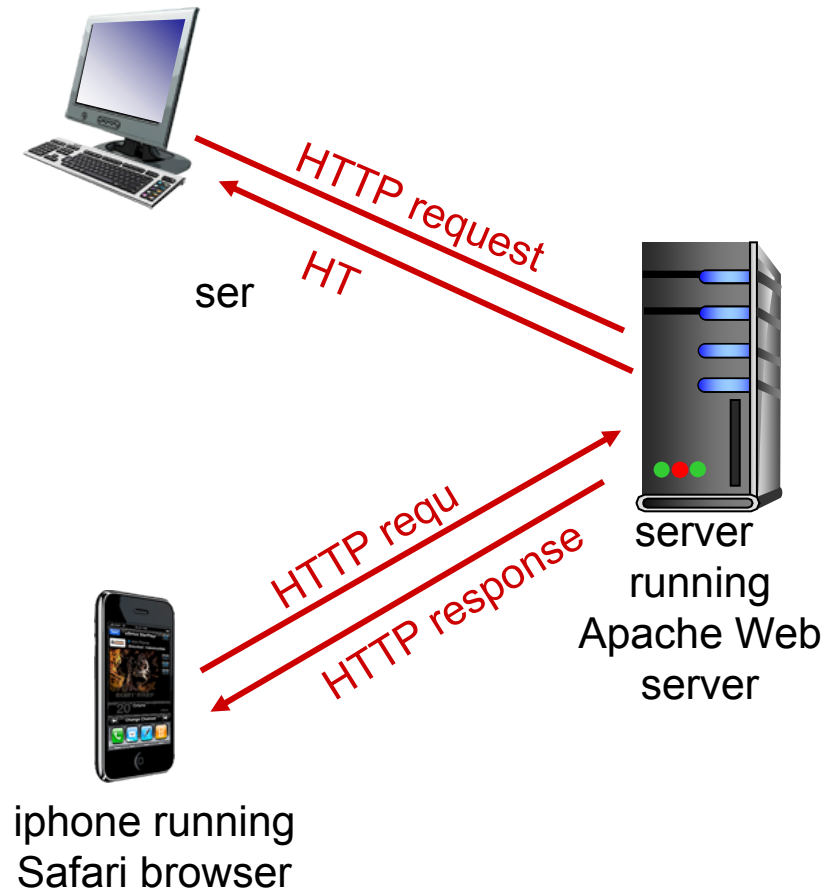
HTTP overview

HTTP: hypertext transfer protocol

- ❖ Web's application layer protocol

tha
receives, (using
HTTP protocol)
and "displays"
Web objects

- **server:** Web server sends (using HTTP protocol) objects in response to requests



HTTP overview (continued)

uses TCP:

- ❖ client initiates TCP connection (creates socket) to serv
- ❖
- ❖ conne
- ❖ HTTP messages (application-layer protocol messages) exchanged between browser (HTTP client) and Web server (HTTP server)
- ❖ TCP connection closed

HTTP is "stateless"

- ❖ server maintains information

maintain "state" are complex!

- ❖ past history (state) must be maintained
- ❖ if server/client crashes, their views of "state" may be inconsistent, must be reconciled

HTTP connections

non-persistent HTTP

- ❖ at most one

- connection closed

- ❖ downloading multiple objects required multiple connections

persistent HTTP

- ❖ multiple objects sent over

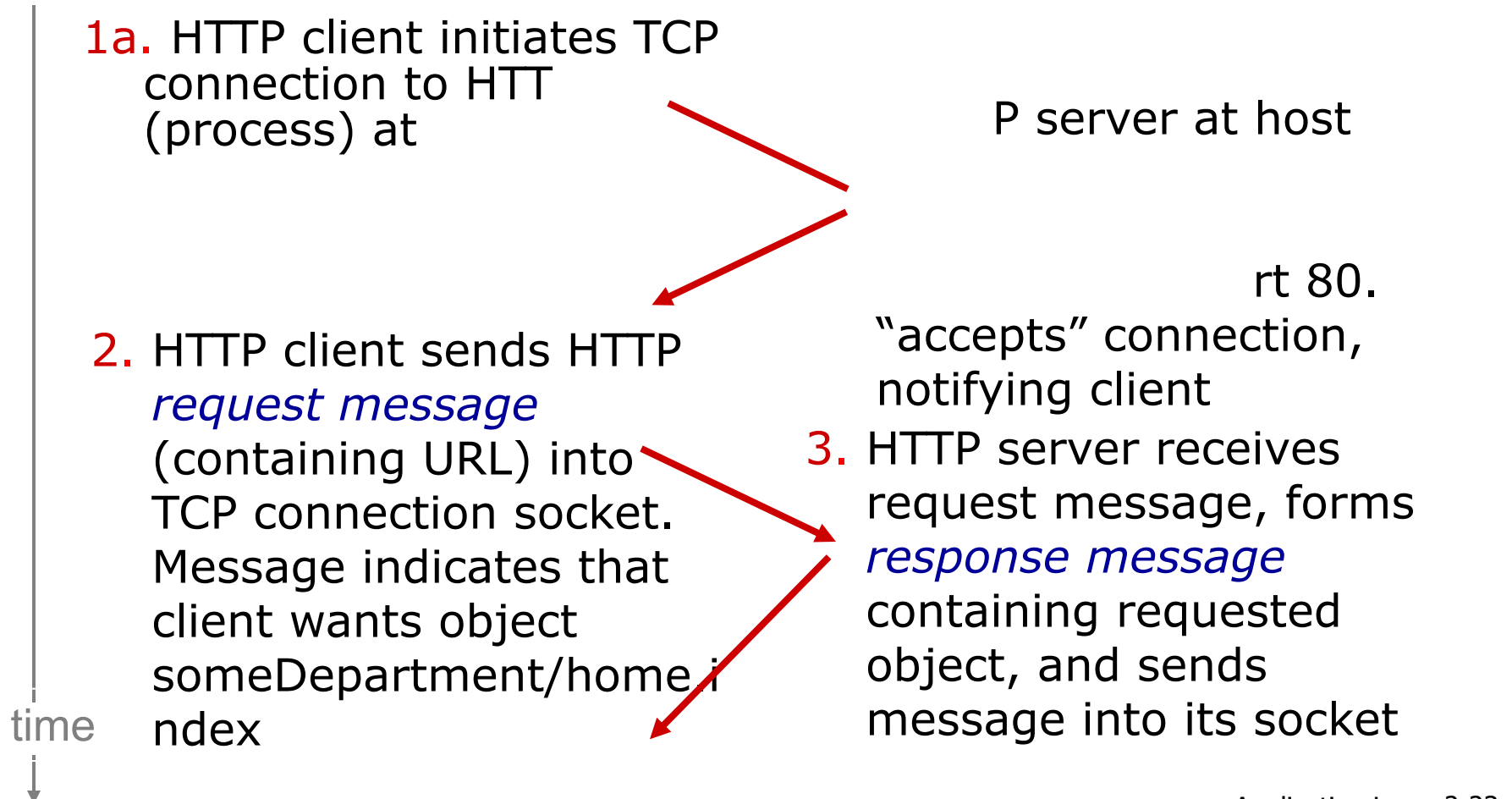
server t,

Non-persistent HTTP

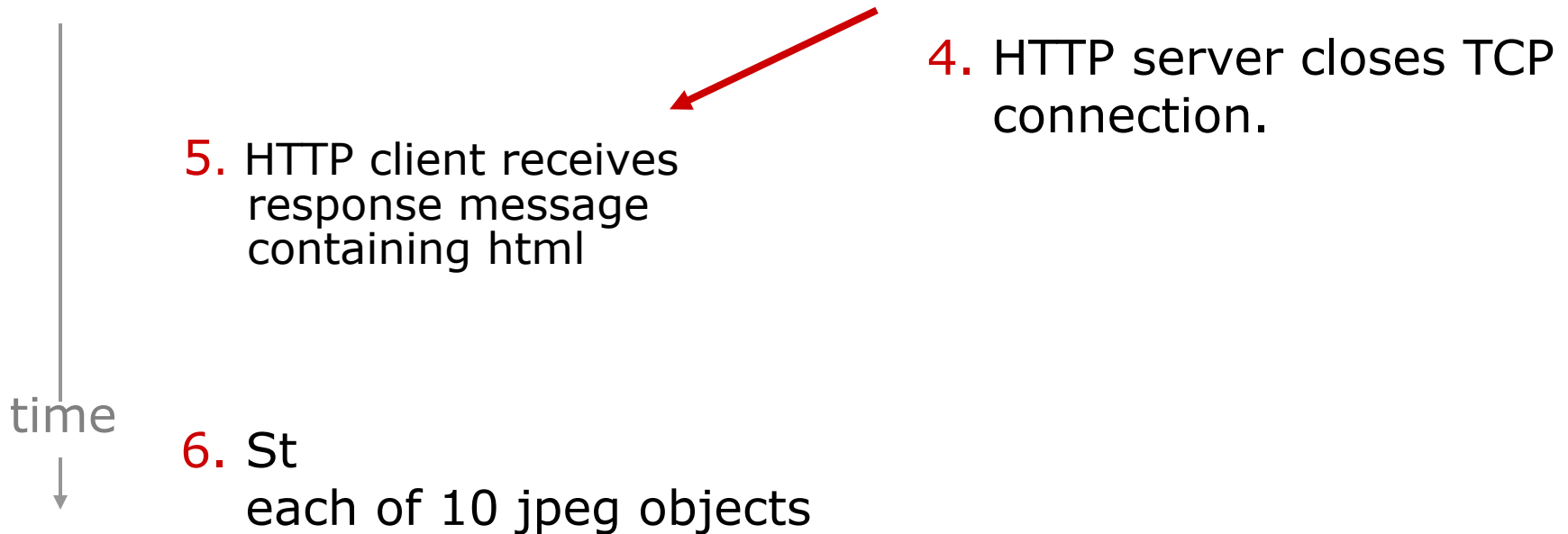
suppose user enters URL:

`www.someSchool.edu/someDepartment/home.index`

(contains text,
references to 10
jpeg images)



Non-persistent HTTP (cont.)

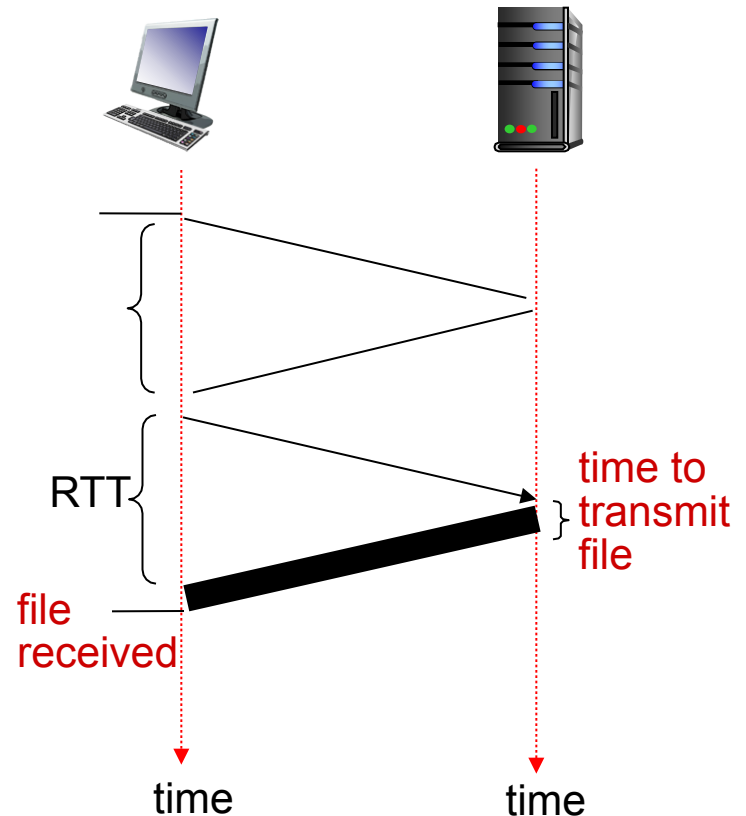


Non-persistent HTTP: response time

RTT (definition): time for a small packet to travel from client to server and back

HTTP response time

- ❖ one RTT for request and first few bytes of HTTP response to return
- ❖ file transmission time
- ❖ non-persistent HTTP response time = $2\text{RTT} + \text{file transmission time}$



Persistent HTTP

non-persistent HTTP issues:

- ❖ requires 2 RTTs per object
- ❖
- ❖ browses parallel TCP connections to fetch referenced objects

persistent HTTP:

- ❖ server leaves connection open sending
- ❖ client sends requests as soon as it encounters a referenced object
- ❖ as little as one RTT for all the referenced

HTTP request message

- ❖ two types of HTTP messages: *request*, *response*

- ❖ HTTP request m

ASCII (human-

req
(GET, POST,
HEAD commands)

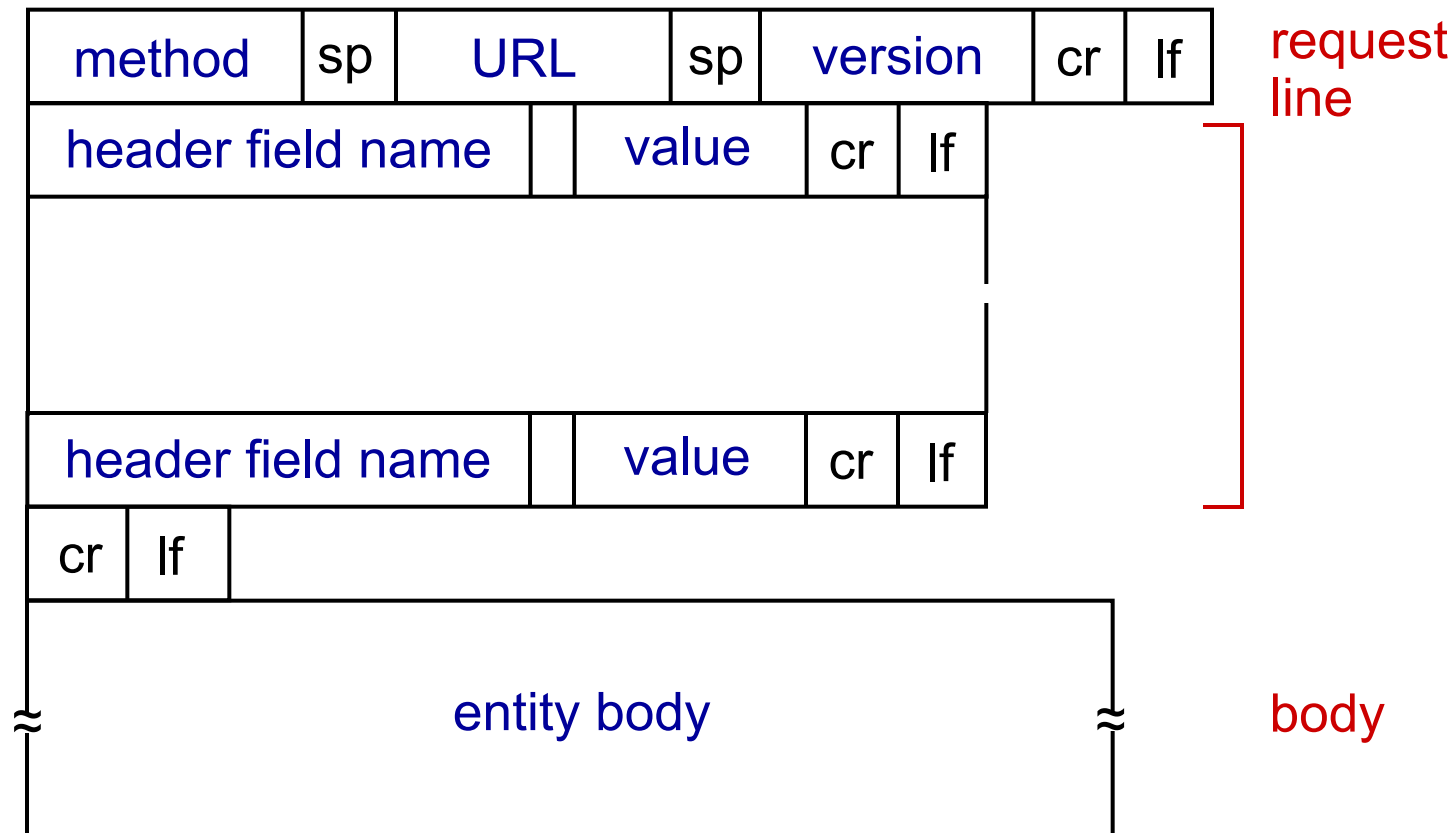
header
lines

carriage return,
line feed at start
of line indicates
end of header lines

Host: www-net.cs.umass.edu\r\n
User-Agent: Firefox/3.6.10\r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Accept-Charset: ISO-8859-1,utf-8;q=0.7\r\n
Keep-Alive: 115\r\n
Connection: keep-alive\r\n
\r\n

carriage return character

HTTP request message: general format



Uploading form input

POST method:

- ❖ web page often includes form input
- ❖ input is upload

URL

- ❖ uses GET method
- ❖ input is uploaded in URL field of request line:

`www.somesite.com/animalsearch?monkeys&banana`

Method types

HTTP/1.0:

- ❖ GET
- ❖ POST

leaves
object out of
response

HTTP/1.1:

- ❖ GET, POST, HEAD

defined in
URL field

- ❖ DELETE
 - deletes file specified in the URL field

HTTP response message

status line
(protocol
status code
status phrase)

```
HTTP/1.1 200 OK\r\n
Date: Sun, 26 Sep 2010 20:09:20 GMT\r\n
Server:                ntOS)\r\n
```

header
lines

```
Content-Length: 2652\r\n
Keep-Alive: timeout=10, max=100\r\n
Connection: Keep-Alive\r\n
Content-Type: text/html; charset=ISO-8859-
1\r\n
\r\n
```

data, e.g.,
requested
HTML file

```
data data data data data ...
```

HTTP response status codes

- ❖ status code appears in 1st line in server-to-client response message.
- ❖ some sample codes:

200 OK

g

301 M

- requested object moved, new location specified later in this msg (Location:)

400 Bad Request

- request msg not understood by server

404 Not Found

- requested document not found on this server

505 HTTP Version Not Supported

Trying out HTTP (client side) for yourself

1. Telnet to your favorite Web server:

```
telnet cis.poly.edu 80
```

opens TCP connection to port 80
server port) at cis.poly.edu.

2. type in a GET HTTP request:

```
GET /~ross/ HTTP/1.1  
Host: cis.poly.edu
```

by typing this in (hit carriage
return twice), you send
this minimal (but complete)
GET request to HTTP server

3. look at response message sent by HTTP server!

(or use Wireshark to look at captured HTTP request/response)

User-server state: cookies

many Web sites use cookies

four components:

- me
- 2) cookie header line in next HTTP *request* message
- 3) cookie file kept on user's host, managed by user's browser
- 4) back-end database at Web

example:

- ❖ Susan always access Internet from PC
pecific e-

- P
- requests arrives at site, site creates:
- unique ID
 - entry in backend database for ID

Cookies: keeping "state" (cont.)

client



server

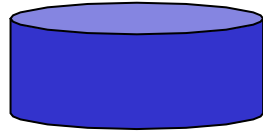


cookie file

usual http request msg

Amazon server
eates ID

usual



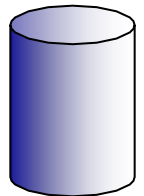
cookie: 1678

cookie-
specific
action

access

usual http response msg

nd
ase



one week later:



ebay 8734
amazon 1678

usual http request msg
cookie: 1678

cookie-
specific
action

access

usual http response msg

Cookies (continued)

what cookies can be used for:

- ❖ authorization
- ❖ shopping carts

(Web

cookies and privacy. ^{aside}

- ❖ cookies permit sites to learn a lot about

how to keep "state":

- ❖ protocol endpoints: maintain state at sender/receiver over multiple transactions
- ❖ cookies: http messages carry state

Web caches (proxy server)

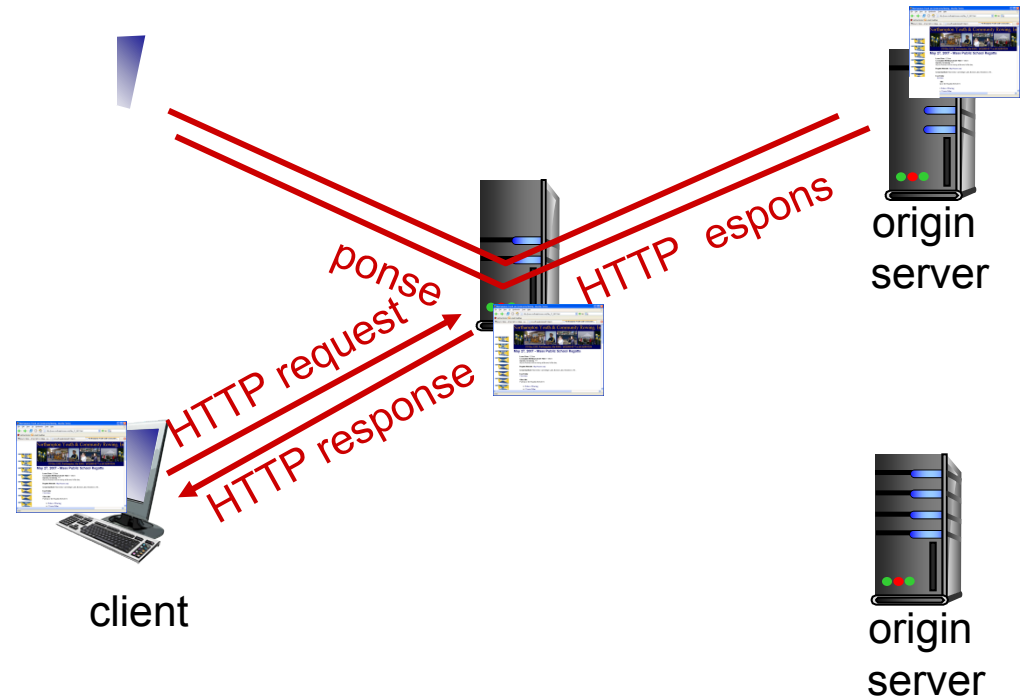
goal: satisfy client request without involving

- ❖ origin server
user sets browser:
Web accesses via
cache



cache

- object in cache:
cache returns
object
- else cache
requests object
from origin server,
then returns
object to client



More about Web caching

- ❖ cache acts as both client and server
 - clie
- ❖ typically cache is installed by ISP (university, company, residential ISP)

why Web caching?

- ❖ reduce response for client

ccess

link

- ❖ Internet dense with caches: enables “poor” content providers to effectively deliver content (so too does P2P file

Caching example:

assumptions:

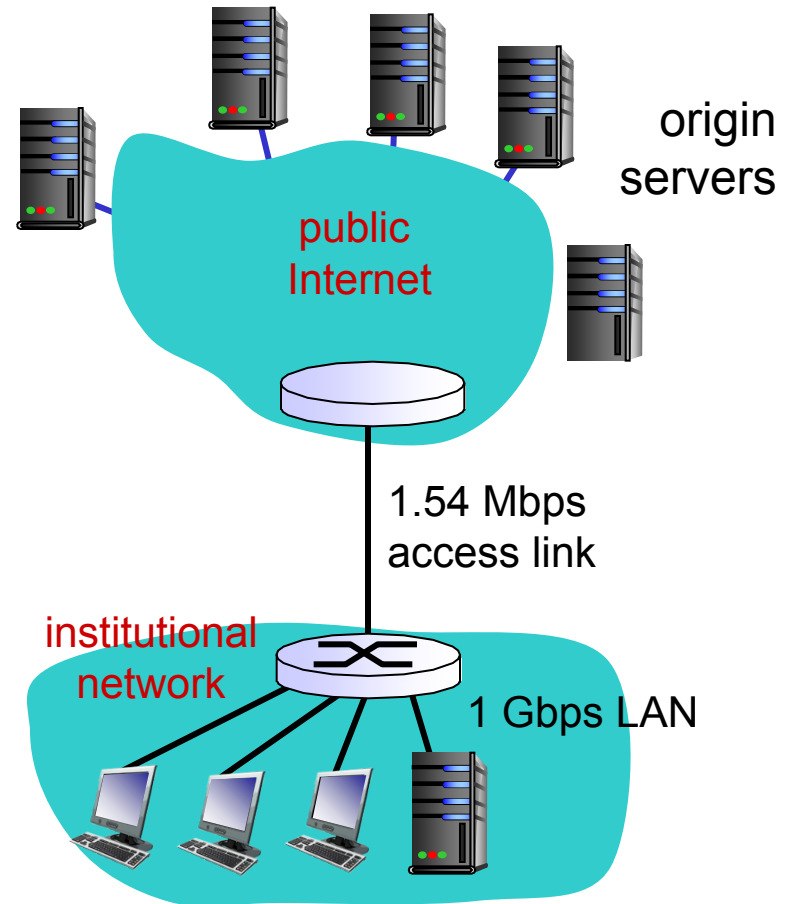
- ❖ avg object size: 100K bits
- ❖ avg request rate from browsers to origin servers: 15/sec

❖

- ❖ RTT from to any origin server: 2 sec
- ❖ access link rate: 1.54 Mbps

consequences: *problem!*

- ❖ LAN utilization: 15%
- ❖ access link utilization = 99%
- ❖ total delay = Internet delay + access delay + LAN delay
= 2 sec + minutes + usecs



Caching example: fatter ~~access link~~

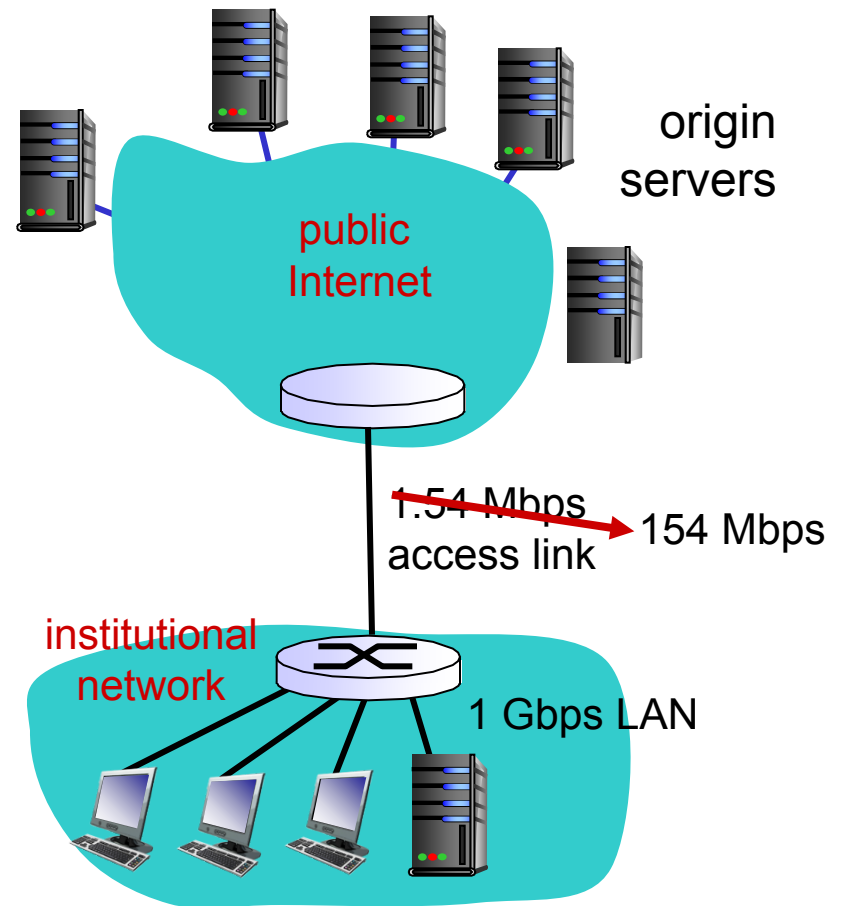
assumptions:

- ❖ avg object size: 100K bits
- ❖ avg request rate from browsers to origin servers: 15/sec
- ❖
- ❖ RTT from to any origin server: 2 sec
- ❖ access link rate: 1.54 Mbps

consequences:

- ❖ LAN utilization: 15%
- ❖ access link utilization = 99%
- ❖ total delay = Internet delay + access delay + LAN delay
= 2 sec + minutes + usecs

Cost: increased access link speed (not cheap!)



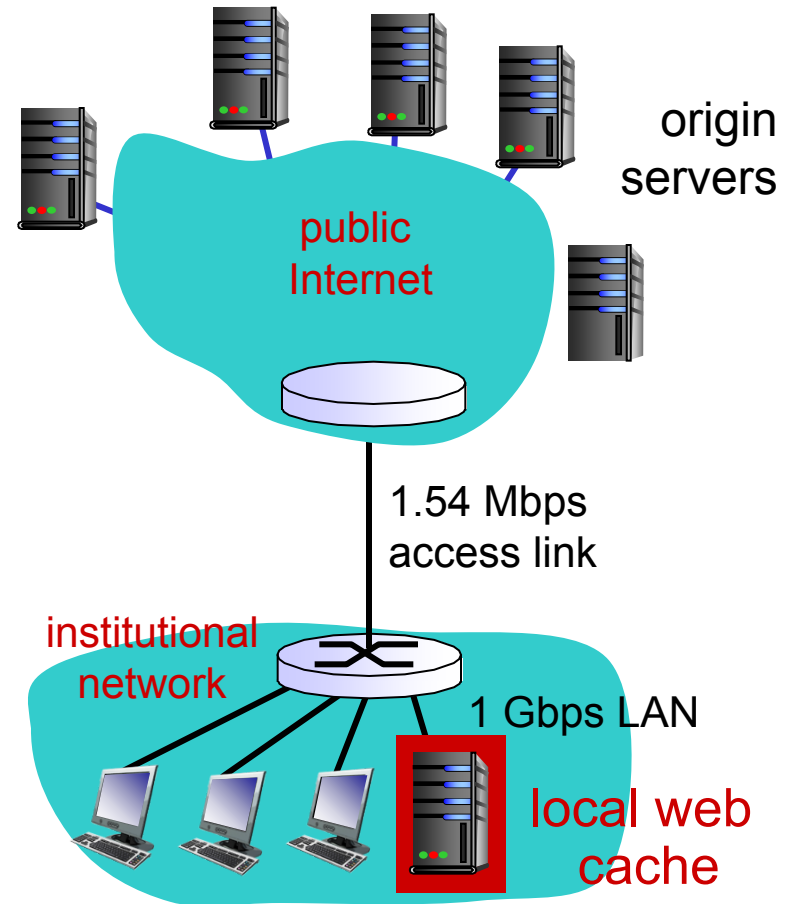
Caching example: install local cache

assumptions:

- ❖ avg object size: 100K bits
- ❖ avg request rate from browsers to origin servers: 15/sec
- ❖
- ❖ RTT from to any origin server: 2 sec
- ❖ access link rate: 1.54 Mbps

consequences: ?

- ❖ LAN utilization ?
- ❖ *How to compute link utilization, delay?*
- ❖ access delay + LAN delay
= 2 sec + minutes + usecs
Cost: web cache (cheap!)

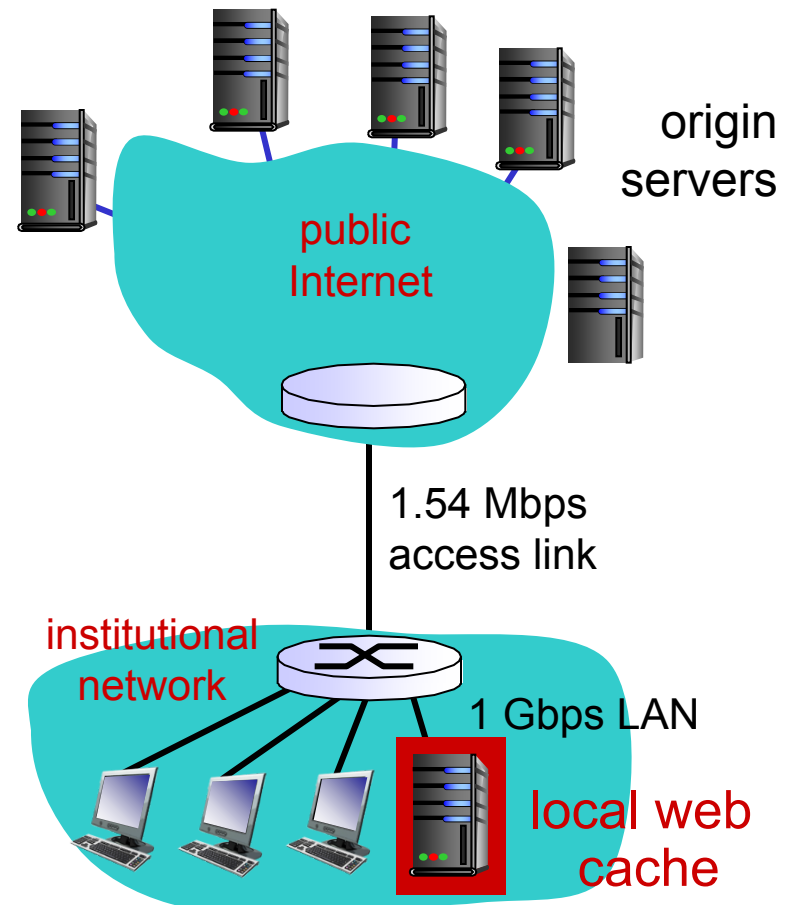


Caching example: install local cache

Calculating access link utilization, delay with cache:

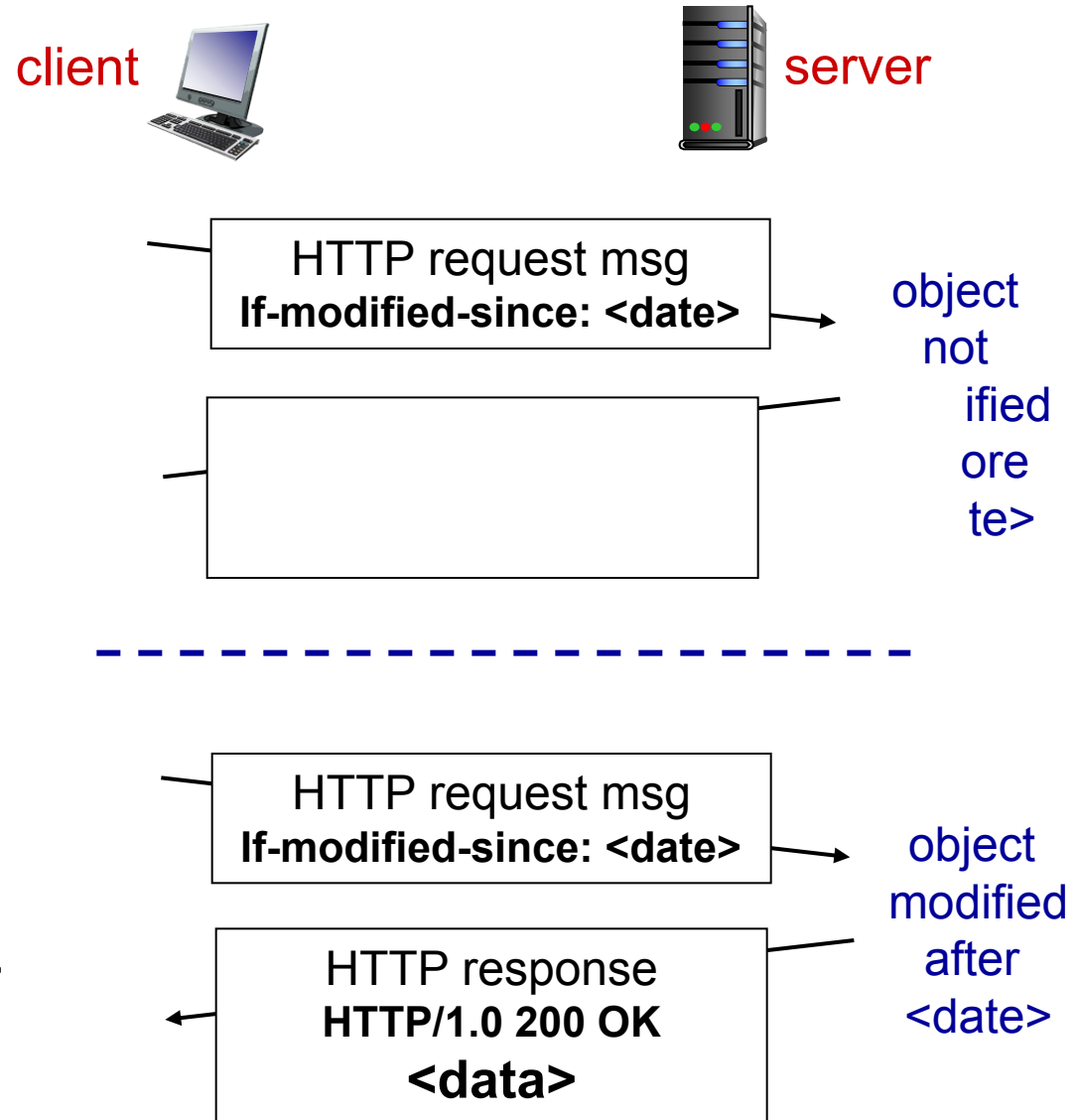
❖ suppose cache h

- cache h is 60% hit
- ❖ data rate to browsers over access link = $0.6 * 1.50 \text{ Mbps} = 0.9 \text{ Mbps}$
- ❖ total delay
 - utilization = $0.9 / 1.54 = .58$
 - = $0.6 * (\text{delay from origin servers}) + 0.4 * (\text{delay when satisfied at cache})$
 - = $0.6 (2.01) + 0.4 (\sim \text{msecs})$
 - = $\sim 1.2 \text{ secs}$
 - less than with 154 Mbps link (and cheaper too!)



Conditional GET

- ❖ *Goal:* don't send object if cache has up-to-date cached version
- ❖ *cache:* of cached copy in HTTP request
`If-modified-since: <date>`
- ❖ *server:* response contains no object if cached copy is up-to-date:
`HTTP/1.0 304 Not Modified`



Chapter 2: outline

2.1 principles of
network
applications

2.6 P2P applications
2.7 socket
gramming

2.2 We

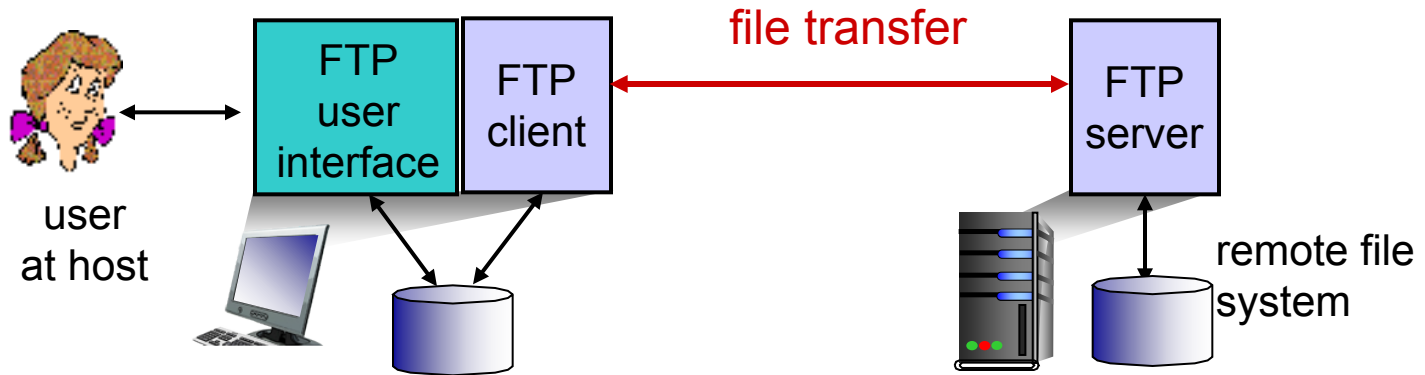
2.3 FTP

2.4 electronic mail

- SMTP, POP3,
IMAP

2.5 DNS

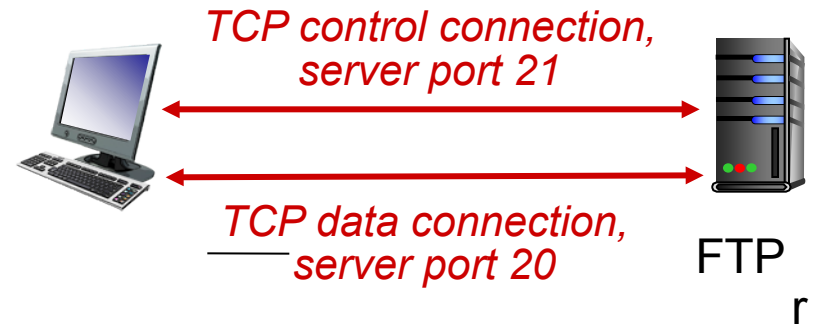
FTP: the file transfer protocol



- ❖ transfer file to/from remote host
- ❖ client/server model
 - *client*: side that initiates transfer (either to/from remote)
 - *server*: remote host
- ❖ ftp: RFC 959
- ❖ ftp server: port 21

FTP: separate control, data connections

- ❖ FTP client contacts FTP server at port 21, using TCP
- ❖ client authorized



- ❖ direct commands over control connection
- ❖ when server receives file transfer command, *server* opens 2nd TCP data connection (for file) to client
- ❖ after transferring one file, server closes data connection
- ❖ no other TCP data connection to transfer another file
- ❖ control connection: *"out of band"*
- ❖ FTP server maintains "state": current directory, earlier authentication

FTP commands, responses

sample commands:

- ❖ sent as ASCII text over control channel
- ❖ **USER *username***

file in
directory

- ❖ **RETR filename**
retrieves (gets) file
- ❖ **STOR filename**
stores (puts) file
onto remote host

sample return codes

- ❖ status code and
ase (as in HTTP)

**connection
already open;
transfer starting**

- ❖ **425 Can't open
data connection**
- ❖ **452 Error writing
file**

Chapter 2: outline

2.1 principles of
network
applications

2.6 P2P applications
2.7 socket
gramming

2.2 We

2.3 FTP

2.4 electronic mail

- SMTP, POP3,
IMAP

2.5 DNS

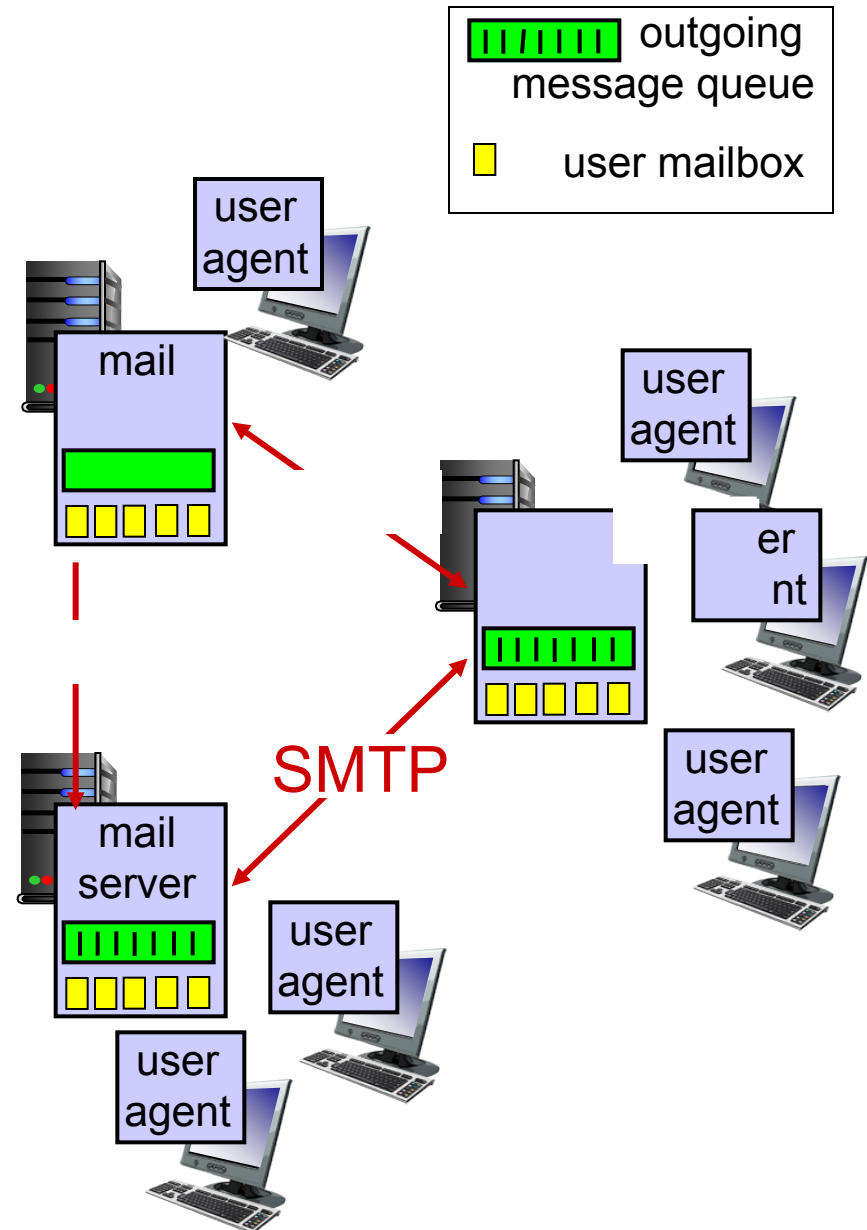
Electronic mail

Three major components:

- ❖ user agents
- ❖ mail servers

User Agent

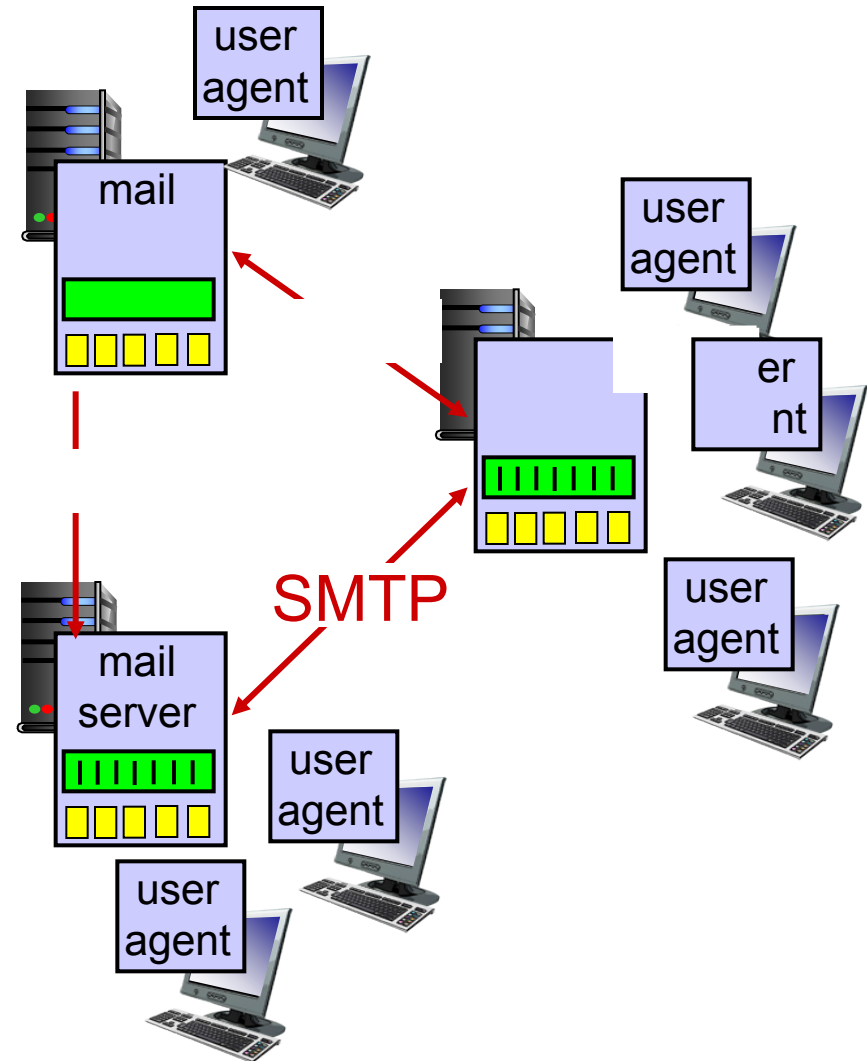
- ❖ a.k.a. "mail reader"
- ❖ composing, editing, reading mail messages
- ❖ e.g., Outlook, Thunderbird, iPhone mail client



Electronic mail: mail servers

mail servers:

- ❖ *mailbox* contains incoming messages for user
- ❖
mail
 - ❖ *SMTP protocol* between mail servers to send email messages
 - client: sending mail server
 - "server": receiving mail server



Electronic Mail: SMTP [RFC 2821]

- ❖ uses TCP to reliably transfer email message from client to server, port 25
- ❖ direct transfer: sending server to
 - ha
 - transfer of messages
 - closure
- ❖ command/response interaction (like HTTP, FTP)
 - **commands**: ASCII text
 - **response**: status code and phrase
- ❖ messages must be in 7-bit ASCII

Scenario: Alice sends message to Bob

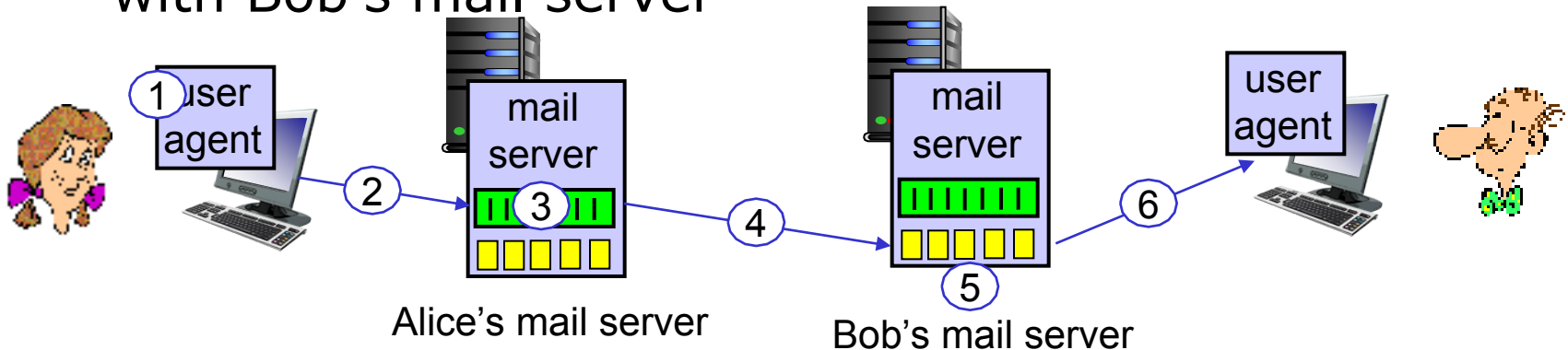
- 1) Alice uses UA to compose message "to" bob@school.edu
- 2) Alice's UA sends message to her

- 4) SMTP client sends Alice's message over the TCP connection
- 5) Bob's mail server the message in

queue

- 3) client side of SMTP opens TCP connection with Bob's mail server

ssage



Sample SMTP interaction

S: 220 hamburger.edu

C: HELO crepes.fr

S: 250 Hello crepes.fr, pleased to meet you

C: MAIL FROM: <alice@crepes.fr>

S: 250 alice@cre ok

C: DATA

S: 354 Enter mail, end with "." on a line by itself

C: Do you like ketchup?

C: How about pickles?

C: .

S: 250 Message accepted for delivery

C: QUIT

S: 221 hamburger.edu closing connection

Try SMTP interaction for yourself:

- ❖ `telnet servername 25`
- ❖ see 220 reply from server
- ❖ enter HELO, M TO, DATA,

above I
client (reader)

ail

SMTP: final words

- ❖ SMTP uses persistent connections
- ❖ SMTP requires message (head

comparison with HTTP:

pull

- ❖ SMTP CRLF.CRLF to determine end of message

I
command/response
interaction, status
codes

- ❖ HTTP: each object encapsulated in its own response msg
- ❖ SMTP: multiple objects sent in multipart msg

Mail message format

SMTP: protocol for
exchanging email
msgs

RFC 822: standar

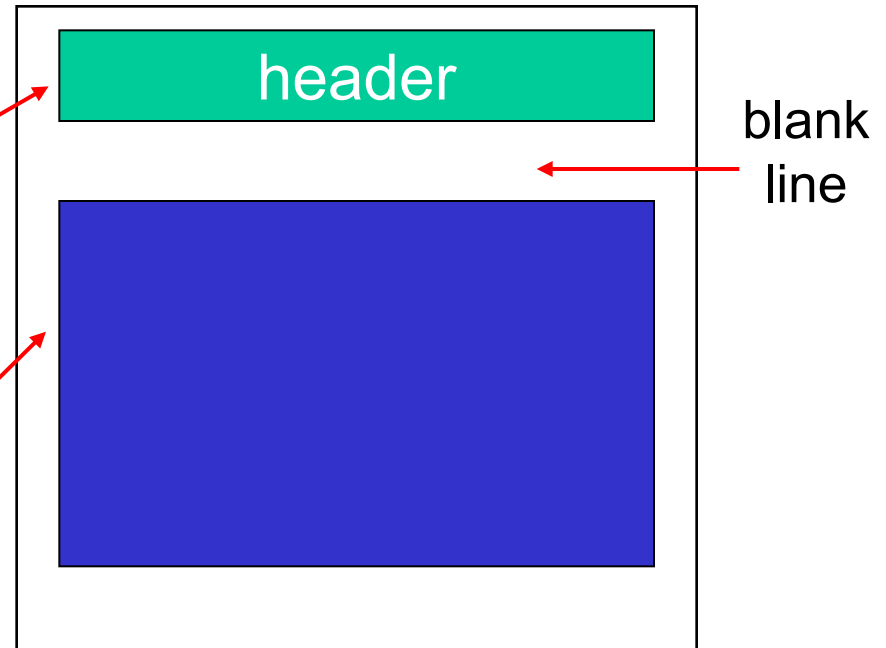
❖ heade

- To:
- From:
- Subject:

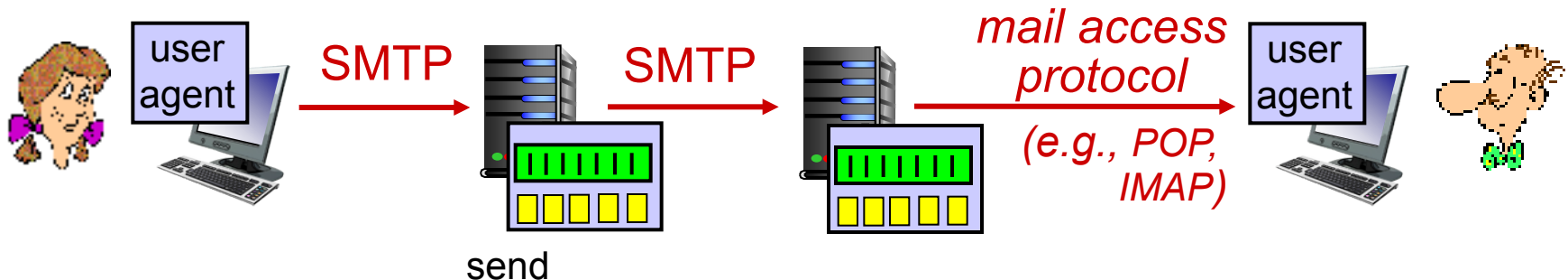
different from SMTP
MAIL FROM, RCPT
TO: commands!

❖ Body: the "message"

- ASCII characters only



Mail access protocols



❖ mail

- **POP:** Post Office Protocol [RFC 1939]: authorization, download
- **IMAP:** Internet Mail Access Protocol [RFC 1730]: more features, including manipulation of stored msgs on server
- **HTTP:** gmail, Hotmail, Yahoo! Mail, etc.

r

POP3 protocol

authorization phase

- ❖ client commands:
 - **user**: declare username

```
S: +OK POP3 server ready
C: user bob
S: +OK
C: pass hungry
S: +OK user successfully logged on
```

- ❖
 - +OK
 - -ERR

transaction phase, client:

- ❖ **list**: list message numbers
- ❖ **retr**: retrieve message by number
- ❖ **dele**: delete
- ❖ **quit**

```
st

S: <message 1 contents>
S: .
C: dele 1
C: retr 2
S: <message 1 contents>
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off
```

POP3 (more) and IMAP

more about POP3

- ❖ previous example uses POP3 “download and read”
- ❖ changes client state
- ❖ POP3 “download-and-keep”: copies of messages on different clients
- ❖ POP3 is stateless across sessions

IMAP

- ❖ keeps all messages in one place: at server
- ❖ keeps user state across sessions:
 - names of folders and mappings between message IDs and folder name

Chapter 2: outline

2.1 principles of
network
applications

2.6 P2P applications

2.7 socket
gramming

2.2 We

2.3 FTP

2.4 electronic mail

- SMTP, POP3,
IMAP

2.5 DNS

DNS: domain name system

people: many identifiers:

- SSN, name, passport #

I

- IP address (32 bit) - used for addressing datagrams
- "name", e.g., www.yahoo.com - used by humans

Q: how to map between IP address and name, and vice

Domain Name System:

- ❖ *distributed database* mented in

e

- ❖ *protocol:* hosts, name servers communicate to *resolve* names (address/name translation)

- note: core Internet function, implemented as application-layer protocol

DNS: services, structure

DNS services

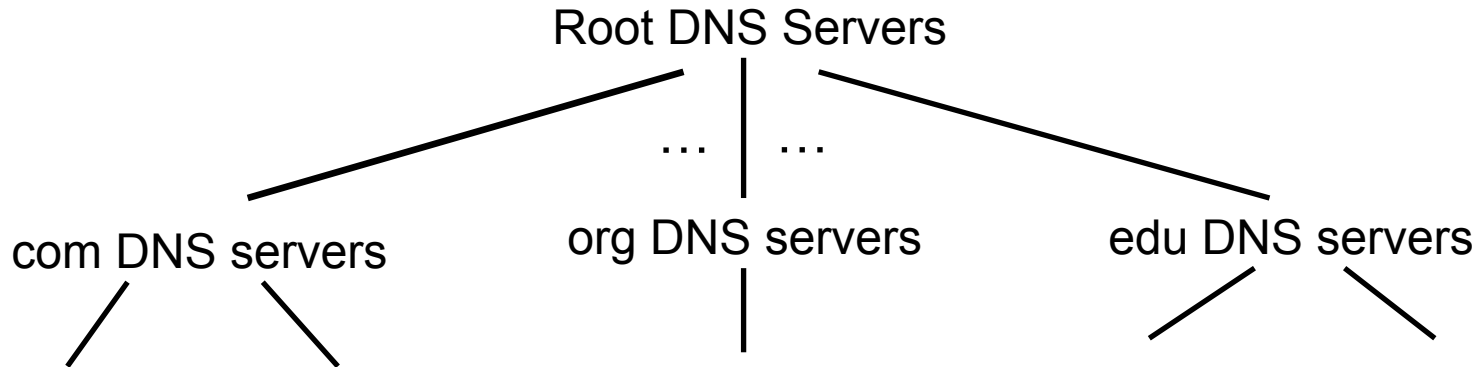
- ❖ hostname to IP address translation
- ❖ host aliasing
- ❖ load
 - replicated Web servers: many IP addresses correspond to one name

why not centralize DNS?

- ❖ single point of failure volume

ale!

DNS: a distributed, hierarchical database



y
D

s

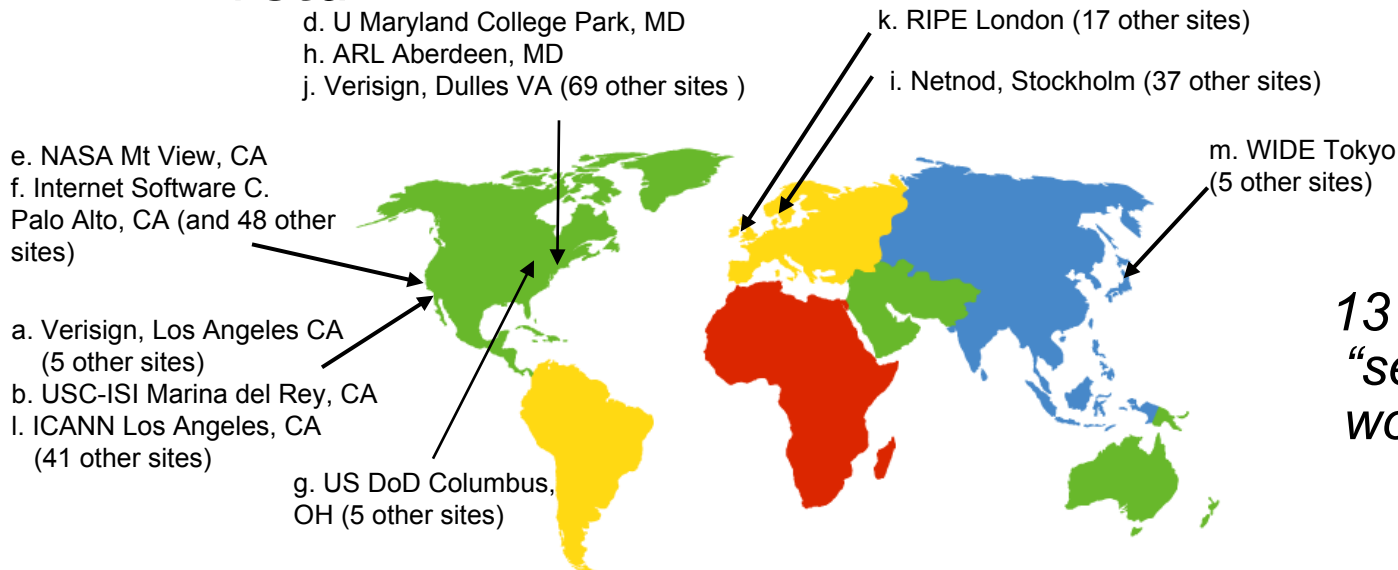
client wants IP for www.amazon.com; 1st approx:

- ❖ client queries root server to find com DNS server
- ❖ client queries .com DNS server to get amazon.com DNS server
- ❖ client queries amazon.com DNS server to get IP address for www.amazon.com

DNS: root name servers

- ❖ contacted by local name server that can not resolve name
- ❖ root name server:
 - contacts autho ver if name mapping

- retu



*13 root name
“servers”
worldwide*

TLD, authoritative servers

top-level domain (TLD) servers:

- responsible for com, org, net, edu, aero, jobs, museums, a ountry domains,

TL

- Educause for .edu TLD

authoritative DNS servers:

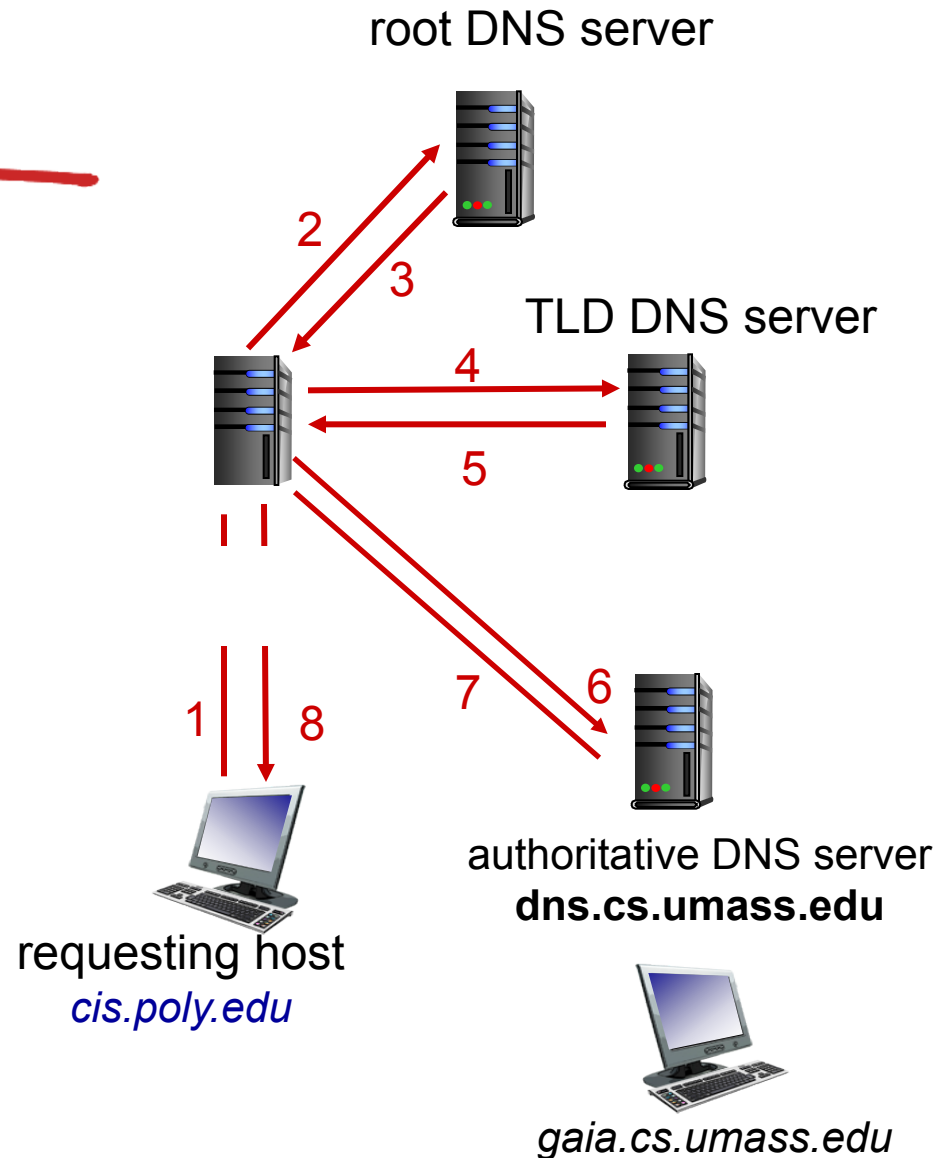
- organization's own DNS server(s), providing authoritative hostname to IP mappings for organization's named hosts
- can be maintained by organization or service provider

Local DNS name server

- ❖ does not strictly belong to hierarchy
- ❖ each ISP (residential ISP, company, university) has
- ❖ sent
 - has local cache of recent name-to-address translation pairs (but may be out of date!)
 - acts as proxy, forwards query into hierarchy

DNS name resolution example

- ❖ host at cis.poly.edu wants IP address for
- ❖ conta replies with name of server to contact
- ❖ "I don't know this name, but ask this server"



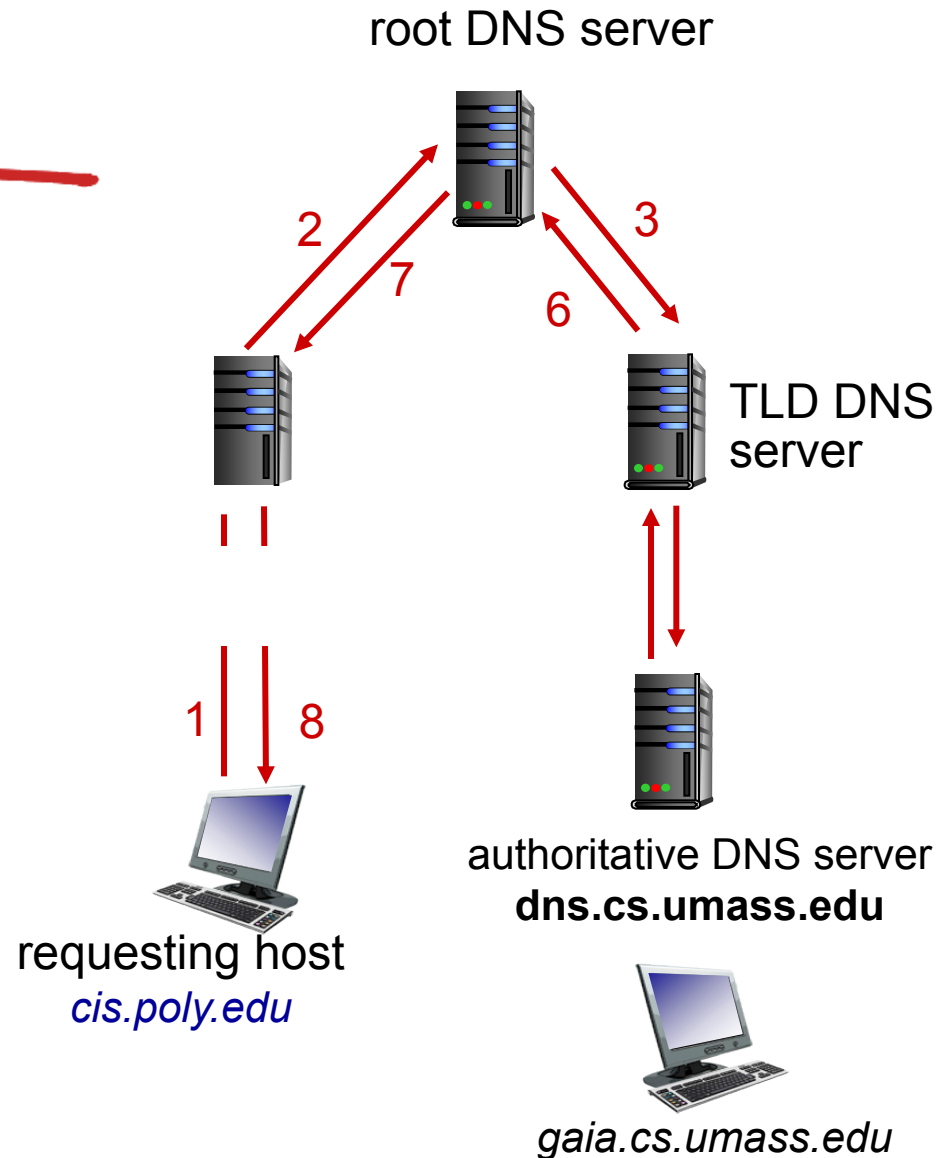
DNS name resolution example

recursive query:

- ❖ puts burden of name resolution

name

- ❖ heavy load at upper levels of hierarchy?



DNS: caching, updating records

- ❖ once (any) name server learns mapping, it *caches* mapping
 - cache entries timeout (disappear) after some time (TTL)
-
- ❖ cached entries may be *out-of-date* (best effort name-to-address translation!)
 - if name host changes IP address, may not be known Internet-wide until all TTLs expire
- ❖ update/notify mechanisms proposed IETF standard
 - RFC 2136

DNS records

DNS: distributed db storing resource records
(RR)

RR format: (name, value, type, ttl)

- **val**

e
eal) name

type=NS

- **name** is domain (e.g., foo.com)
- **value** is hostname of authoritative name server for this domain

- **www.ibm.com** is really **servereast.backup2.ibm.com**
- **value** is canonical name

type=MX

- **value** is name of mailserver associated with **name**

DNS protocol, messages

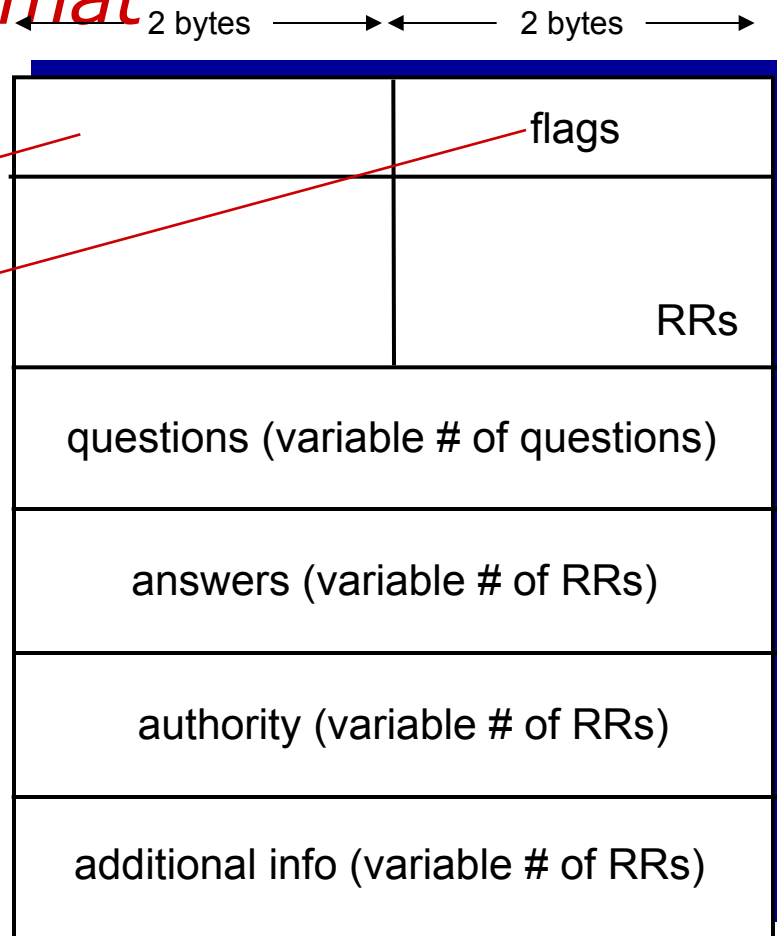
- ❖ *query* and *reply* messages, both with same *message format*

msg header

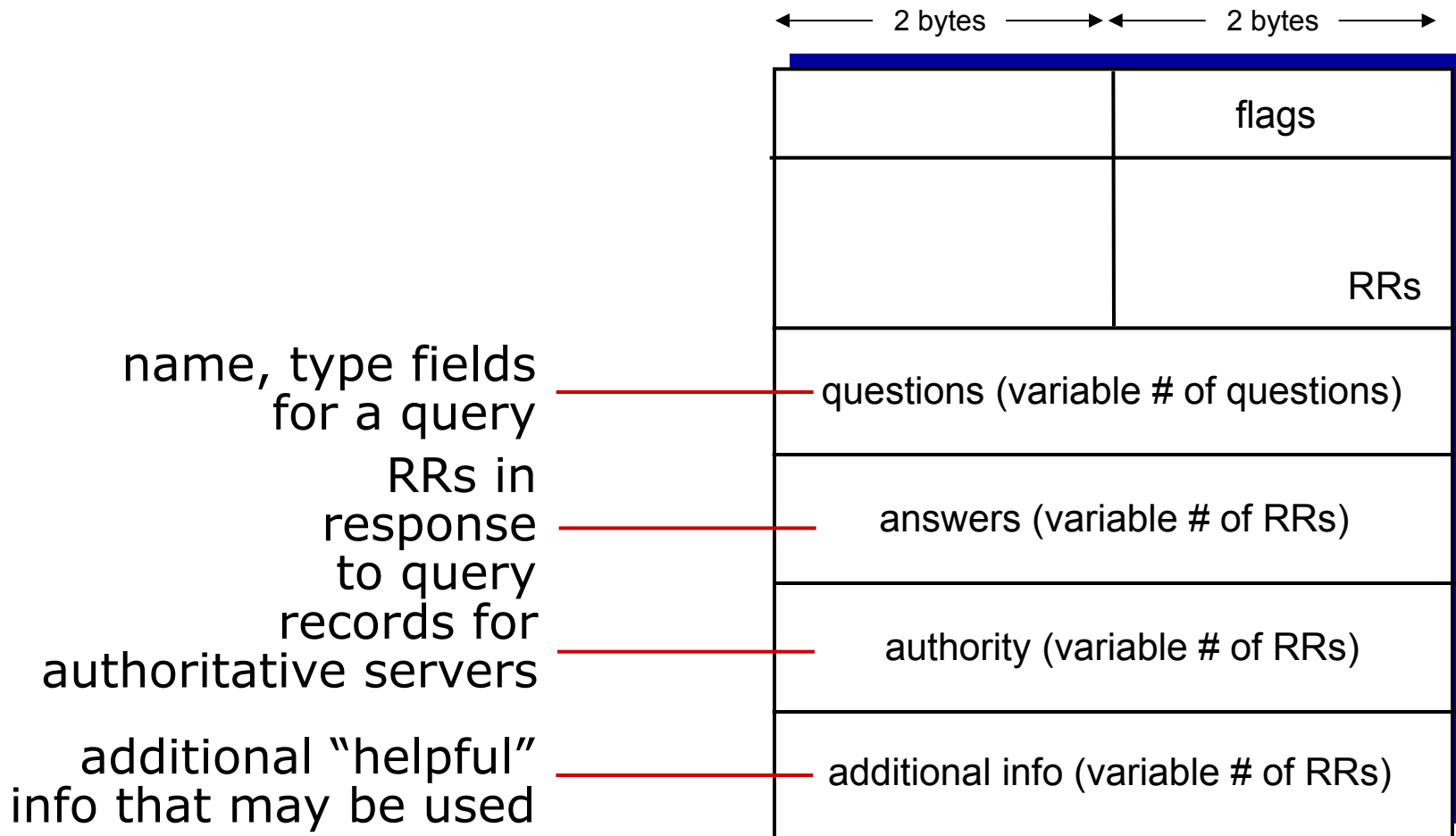
query u

- ❖ **flags:**

- query or reply
- recursion desired
- recursion available
- reply is authoritative



DNS protocol, messages



Inserting records into DNS

- ❖ example: new startup "Network Utopia"
- ❖ register name networkutopia.com at *DNS registrar* (e.g., Network Solutions)
 - provide name of authoritative
 - (networkutopia.com, NS)
(dns1.networkutopia.com, 212.212.212.1, A)
- ❖ create authoritative server type A record for www.networkutopia.com; type MX record for networkutopia.com

Attacking DNS

DDoS attacks

- ❖ Bombard root servers with data
 - Traffic Filtering
 - Local DNS servers cache IPs of TLD servers, allowing root server bypass
- ❖ Bombard TLD servers
 - Potentially more

Redirect attacks

- ❖ Man-in-middle except queries

Exploit DNS for DDoS

- ❖ Send queries with spoofed source address: target IP
- ❖ Requires amplification

Chapter 2: outline

2.1 principles of
network
applications

2.6 P2P applications

2.7 socket
gramming

2.2 We

2.3 FTP

2.4 electronic mail

- SMTP, POP3,
IMAP

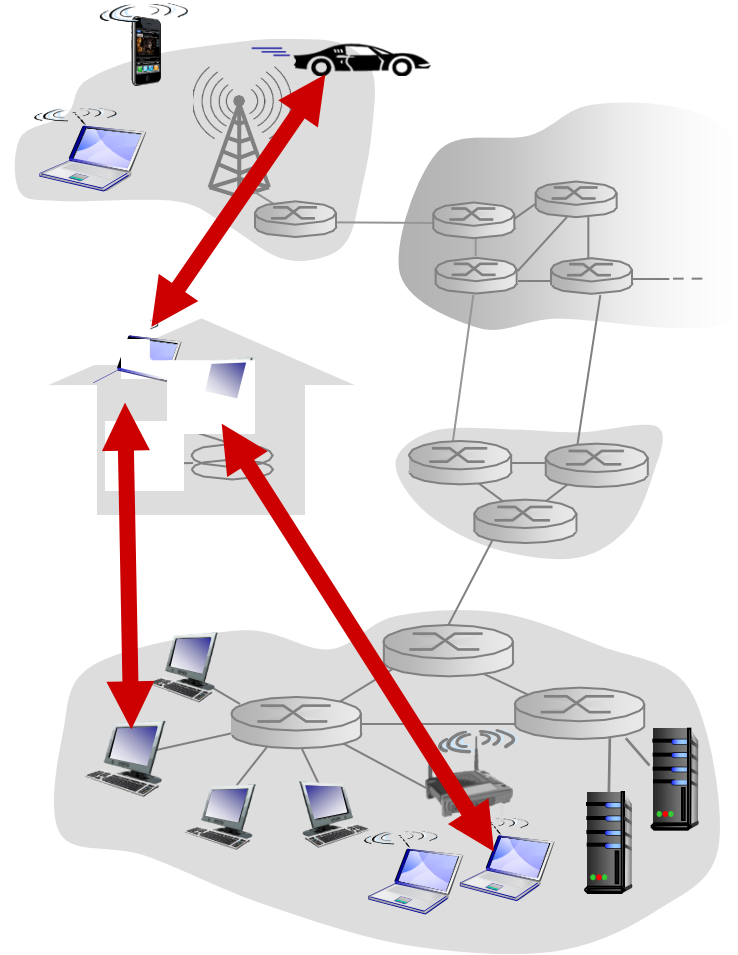
2.5 DNS

Pure P2P architecture

- ❖ *no* always-on server
- ❖ arbitrary end systems directly communicate
- ❖ peers are

examples:

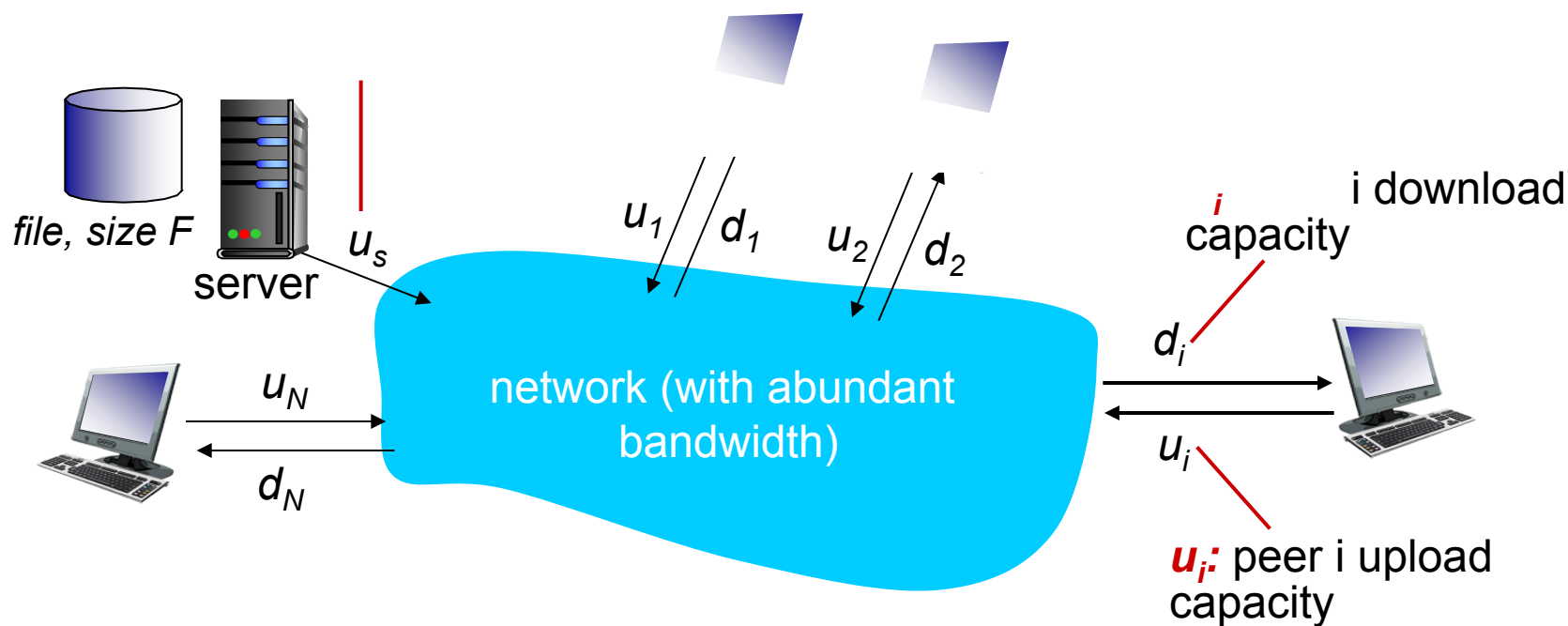
- file distribution (BitTorrent)
- Streaming (KanKan)
- VoIP (Skype)



File distribution: client-server vs P2P

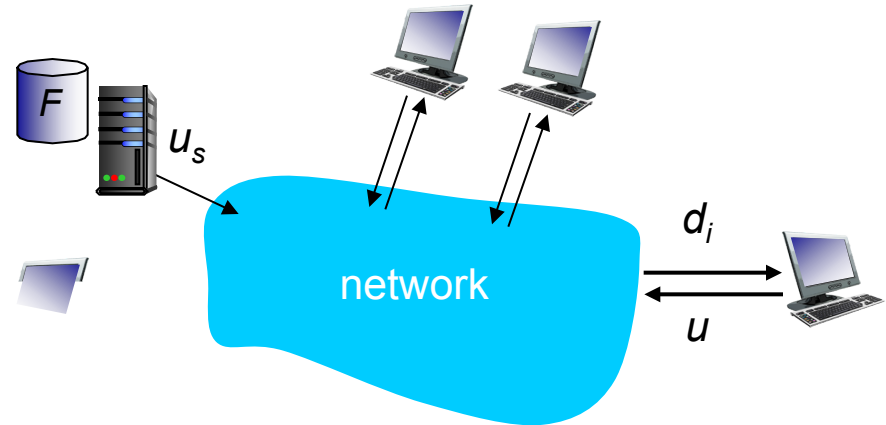
Question: how much time to distribute file (size F) from one server to N peers?

- peer upload/download capacity is limited resource



File distribution time: client-server

- ❖ *server transmission:*
must sequentially send (upload) N file copies:
 - time to send one copy:
 F/u_s



❖ download

- $d_{\min}$ = min client download rate
- min client download time:

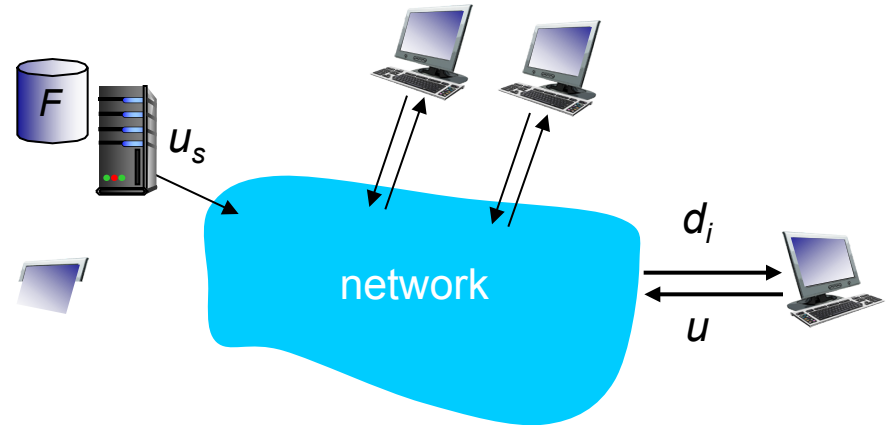
$F/d_{\min}$ time to distribute F
to N clients using
client-server approach

$$D_{c-s} \geq \max\{NF/u_s, F/d_{\min}\}$$

increases linearly in N

File distribution time: P2P

- ❖ **server transmission:**
must upload at least one copy
- ❖ **client:** time to send one copy:
download file co



- ❖ **clients:**
bits
- max upload rate (limiting max download

rate) is $u_s + \sum u_i$

time to distribute F to N clients using P2P approach

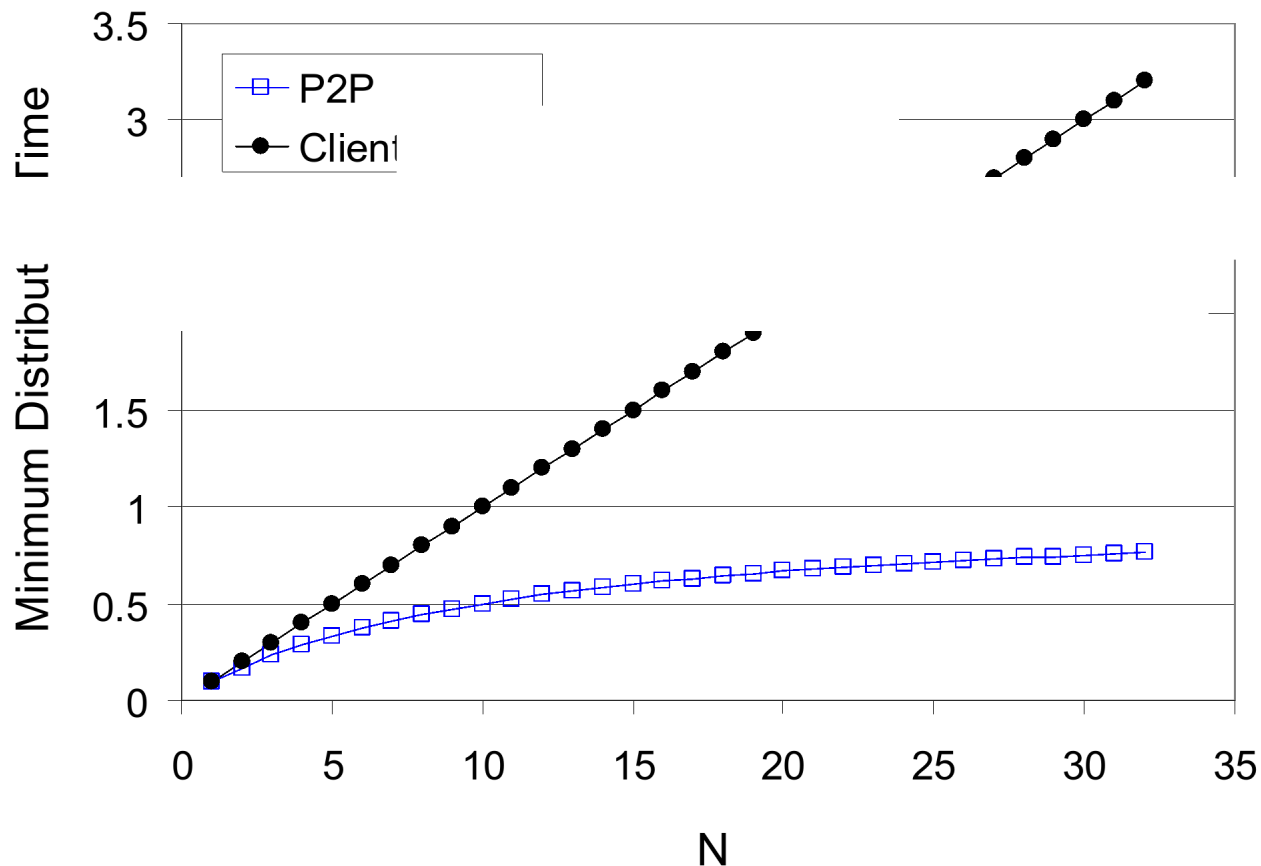
$$D_{P2P} \geq \max\{F/u_s, F/d_{min}, NF/(u_s + \sum u_i)\}$$

increases linearly in N ...

... but so does this, as each peer brings service capacity

Client-server vs. P2P: example

client upload rate = u , $F/u = 1$ hour, $u_s = 10u$, $d_{min} \geq u_s$



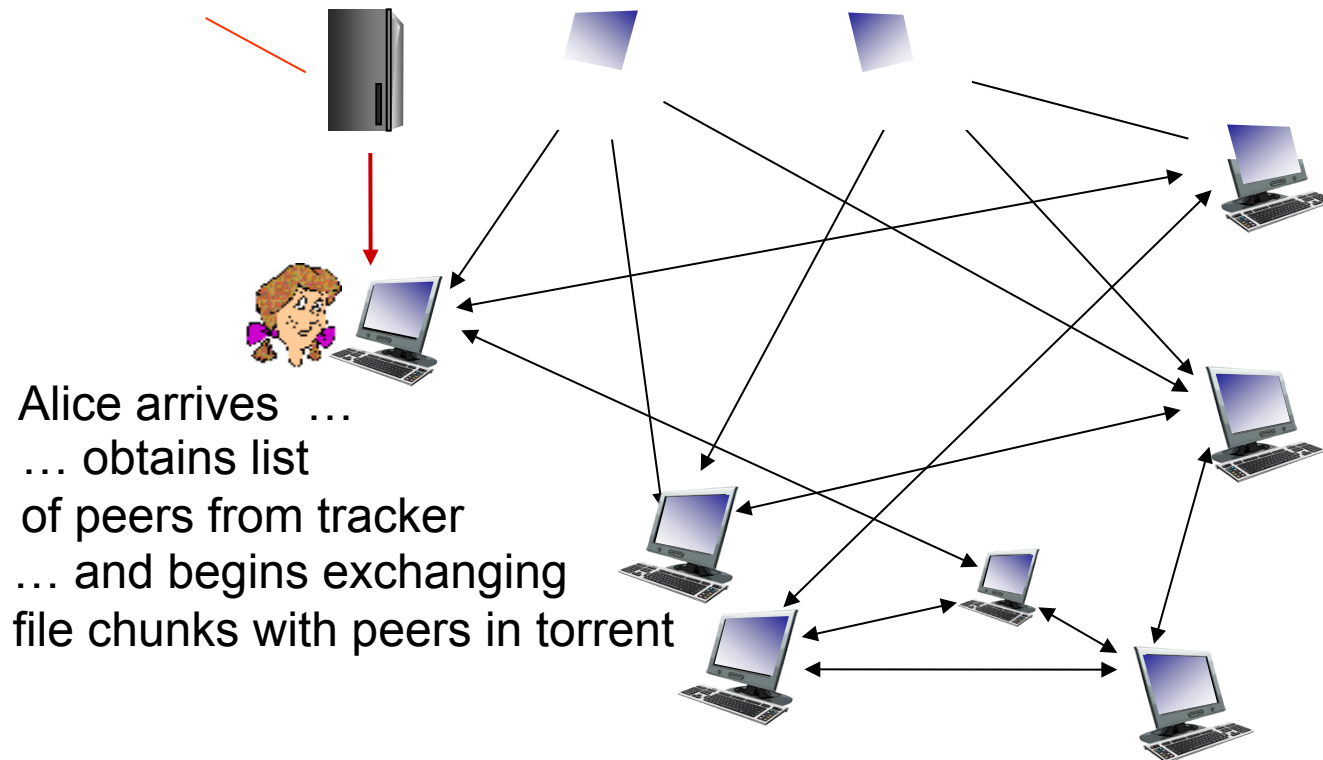
P2P file distribution:

~~BitTorrent~~

- ❖ file divided into 256Kb chunks
- ❖ peers in torrent send/receive file chunks

tracker: tracks peers participating in torrent

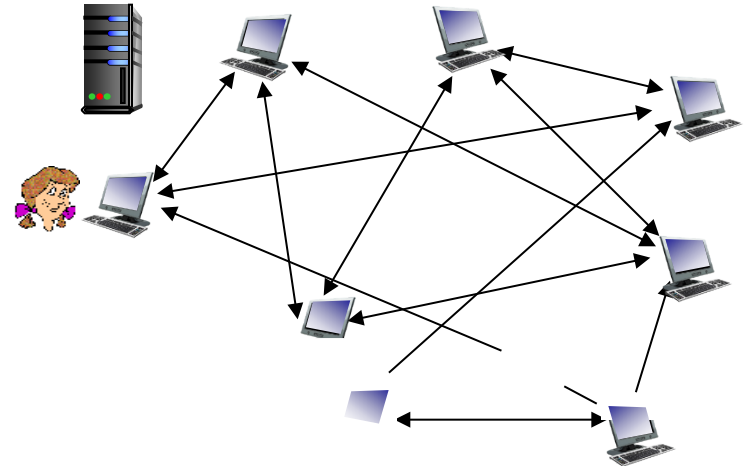
torrent: group of peers exchanging



P2P file distribution:

~~BitTorrent~~

- ❖ peer joining torrent:
 - has no chunks, but will accumulate them over time from oth



con
peers ("neighbors")

- ❖ while downloading, peer uploads chunks to other peers
- ❖ peer may change peers with whom it exchanges chunks
- ❖ *churn*: peers may come and go
- ❖ once peer has entire file, it may (selfishly) leave or (altruistically) remain in torrent

BitTorrent: requesting, sending file chunks

requesting chunks:

- ❖ at any given time, different peers have different subsets

- ❖ each peer keeps track of the chunks that they have
- ❖ Alice requests missing chunks from peers, rarest first

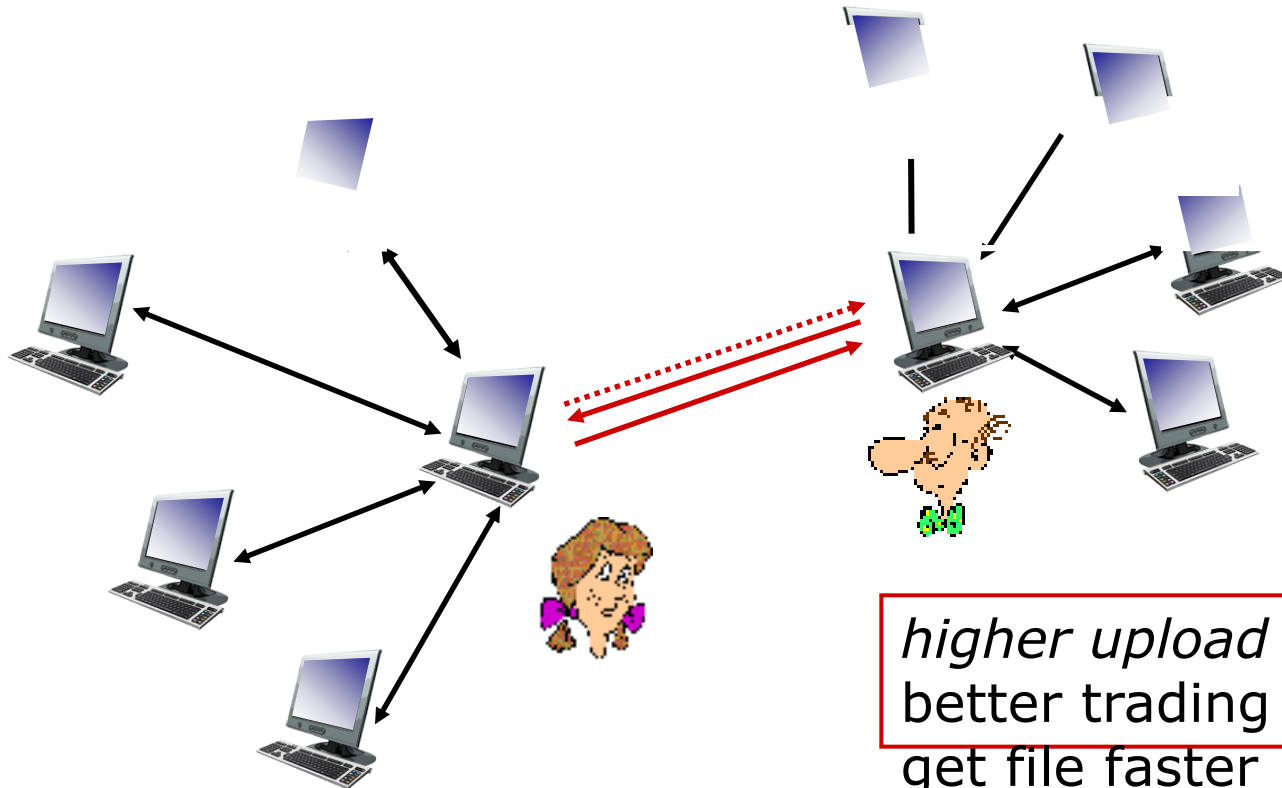
sending chunks: tit-for-tat

ends chunks to

- if a peer is choked by Alice (do not receive chunks from her)
- re-evaluate top 4 every 10 secs
- ❖ every 30 secs: randomly select another peer, starts sending chunks
 - “optimistically unchoke” this peer
 - newly chosen peer may join

BitTorrent: tit-for-tat

- (1) Alice “optimistically unchokes” Bob
- (2) Alice becomes one of Bob’s top-four providers; Bob reciprocates
- (3) Bob becomes one of Alice’s top-four providers



Distributed Hash Table (DHT)

- ❖ Hash table
- ❖ DHT paradigm
- ❖
- ❖ Peer churn

Simple Database

Simple database with (key, value) pairs:

- key: human name; value: social security #

Key	
Xiaoming Liu	385-41-0902
Rakesh Gopal	441-89-1956
Linda Cohen	217-66-5609
.....	
Lisa Kobayashi	177-23-0199

- key: movie title; value: IP address

Hash Table

- More convenient to store and search on numerical representation of key
- $\text{key} = \text{hash}(\text{key})$

Diana		
Xiaoming Liu	1567109	385-41-0902
Rakesh Gopal	2360012	441-89-1956
Linda Cohen	5430938	217-66-5609
.....		
Lisa Kobayashi	9290124	177-23-0199

Distributed Hash Table (DHT)

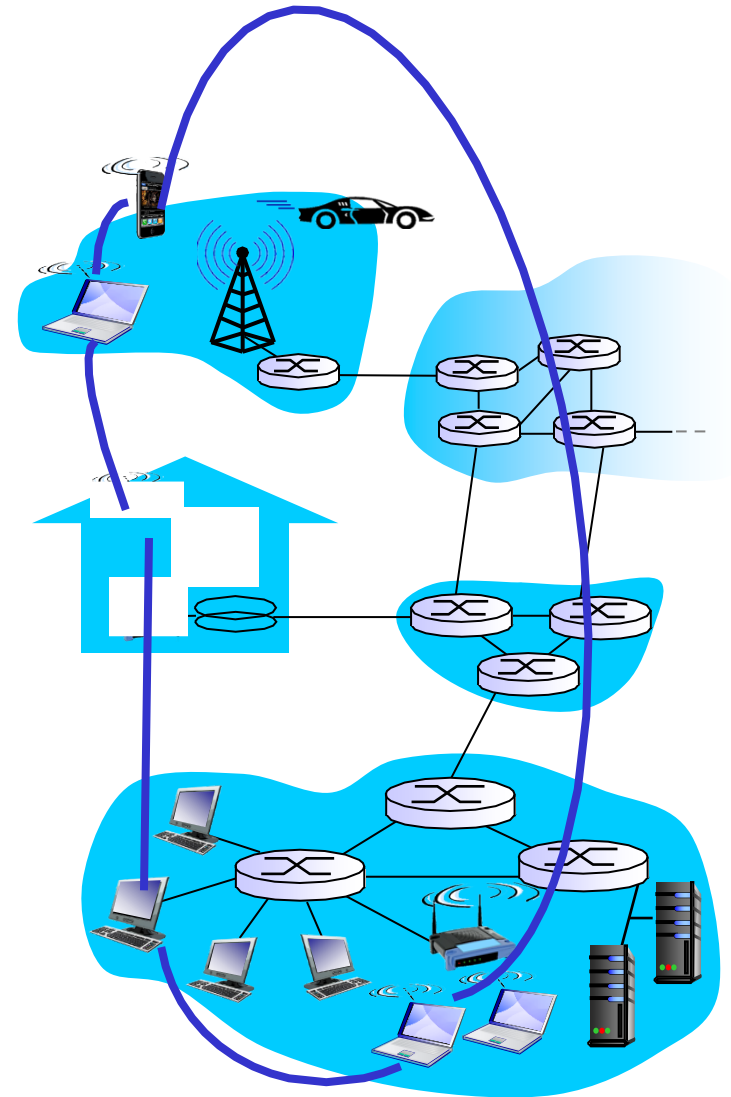
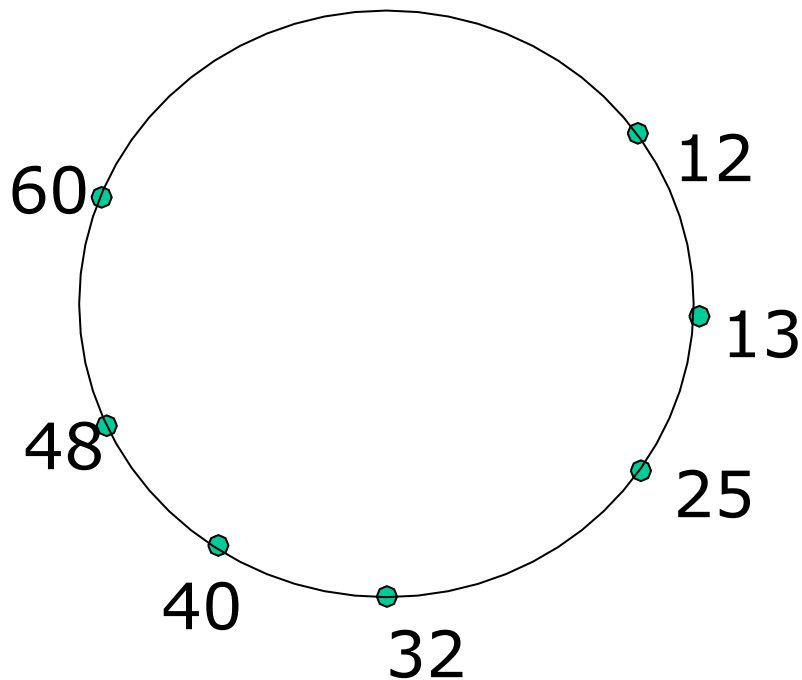
- ❖ Distribute (key, value) pairs over millions of peers
 - pairs are evenly distributed over peers
- ❖ Any peer can exchange with a key
- ❖ Each peer only knows about a small number of other peers
- ❖ Robust to peers coming and going (churn)

Assign key-value pairs to ~~peers~~

- ❖ rule: assign key-value pair to the peer that has the *closest* ID.
- ❖ convention: closest is the *immediate successor* of
- ❖ supp
1,12,13,25,32,40,48,60
 - If key = 51, then assigned to peer 60
 - If key = 60, then assigned to peer 60
 - If key = 61, then assigned to peer 1

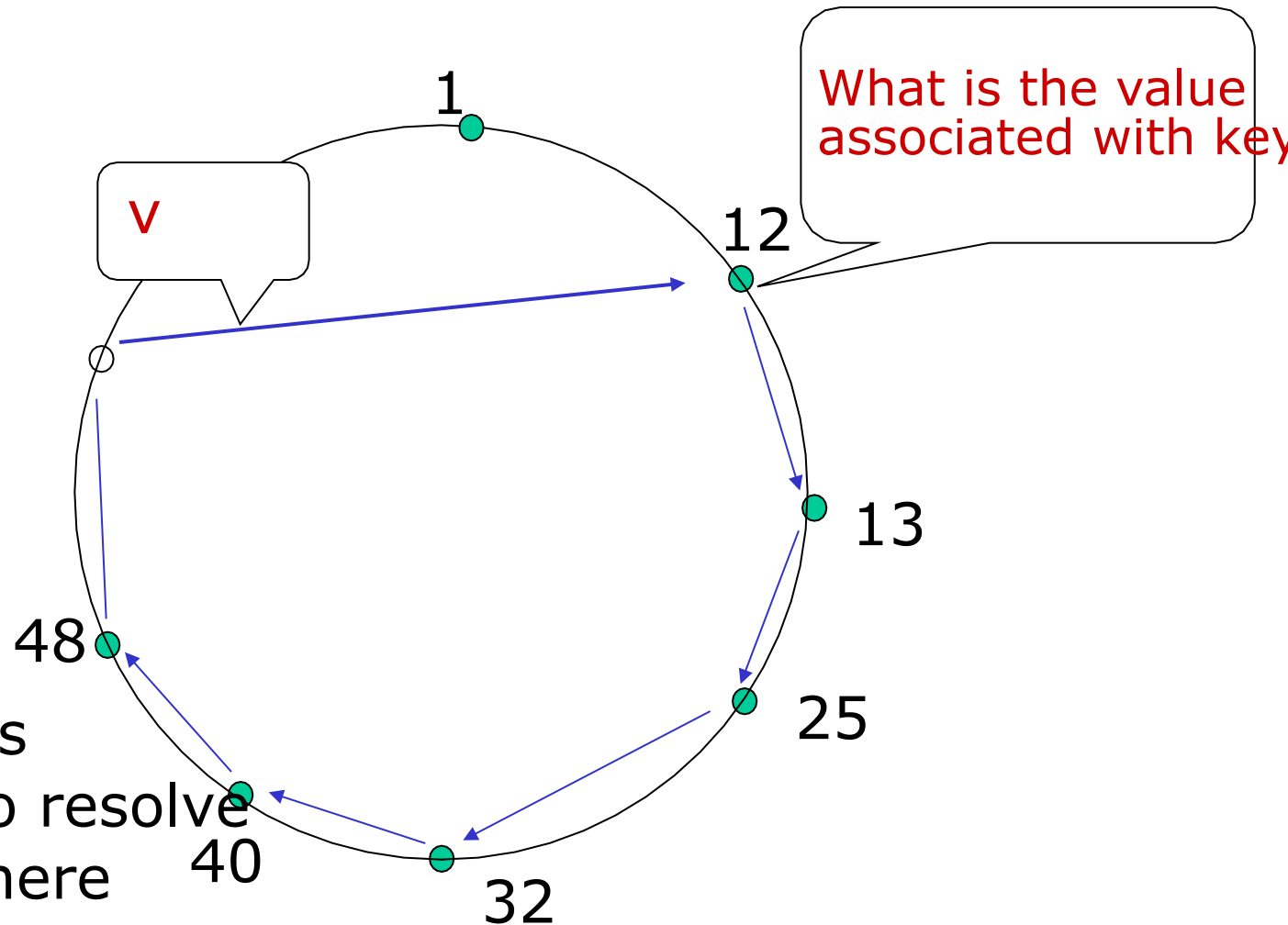
Circular DHT

- each peer *only* aware of immediate successor and predecessor.



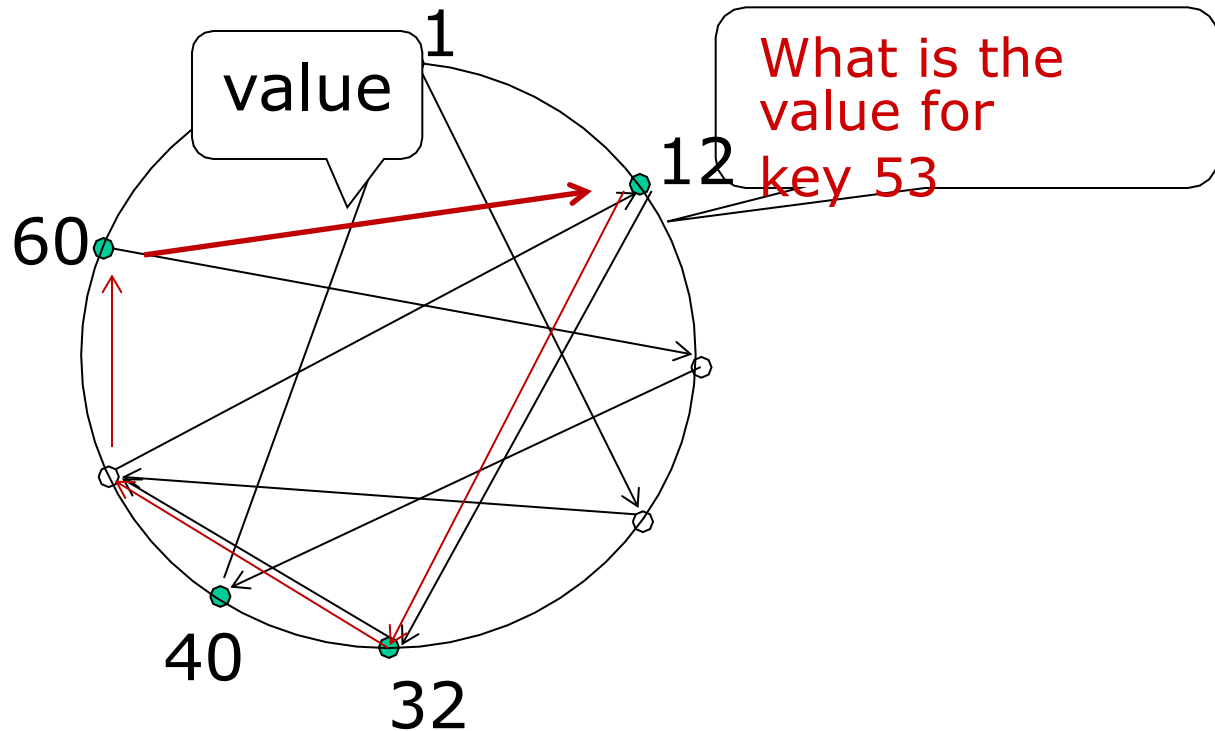
“overlay network”

Resolving a query



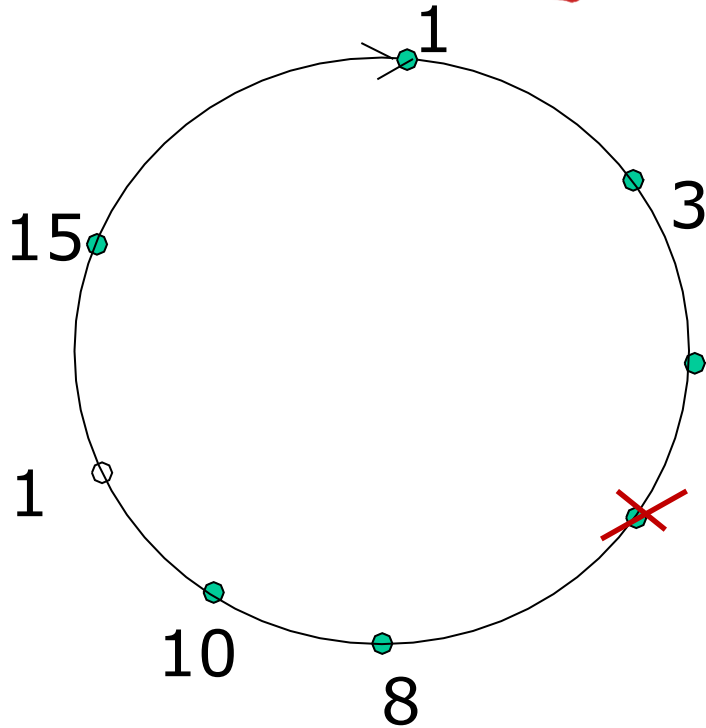
$O(N)$ messages
on average to resolve
query, when there
are N peers

Circular DHT with shortcuts



- each peer keeps track of IP addresses of predecessor, successor, short cuts.
- reduced from 6 to 3 messages.
- possible to design shortcuts with $O(\log N)$ neighbors, $O(\log N)$ messages in query

Peer churn



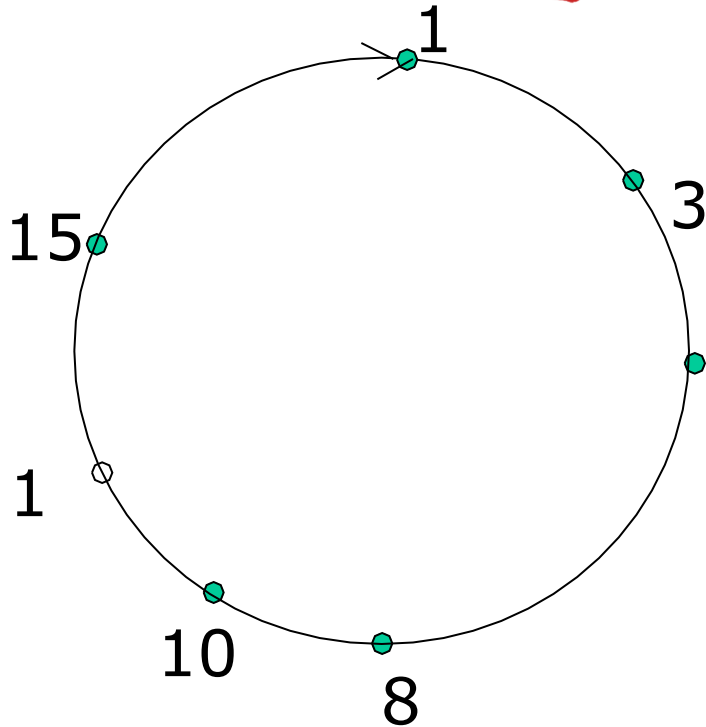
example: peer 5 abruptly leaves

handling peer churn:

- ❖ peers may come and go (churn)
- ❖ each peer knows address of its two successors
peer periodically

- ❖ if immediate successor leaves, choose next successor as new immediate successor

Peer churn



handling peer churn:

- ❖ peers may come and go (churn)
- ❖ each peer knows address of its two successors
- peer periodically

- ❖ if immediate successor leaves, choose next successor as new immediate successor

example: peer 5 abruptly leaves

- ❖ peer 4 detects peer 5's departure; makes 8 its immediate successor
- ❖ 4 asks 8 who its immediate successor is; makes 8's immediate successor its second successor.

Chapter 2: outline

2.1 principles of
network
applications

2.6 P2P applications

2.7 socket
gramming

2.2 We

2.3 FTP

2.4 electronic mail

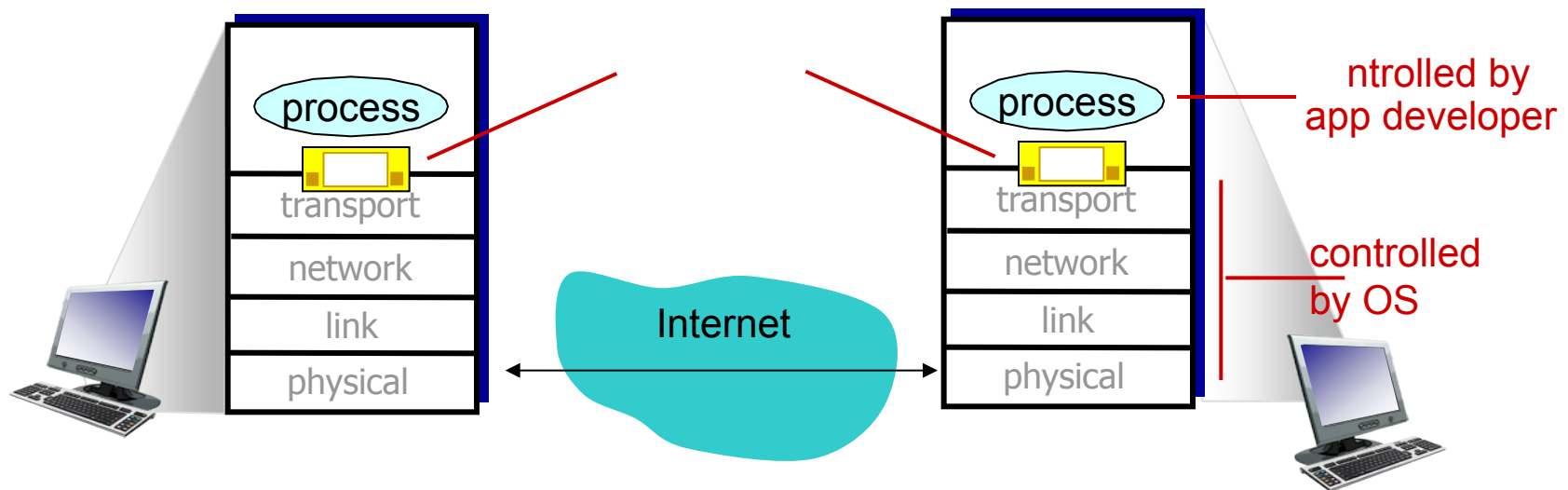
- SMTP, POP3,
IMAP

2.5 DNS

Socket programming

goal: learn how to build client/server applications that communicate using sockets

socket: door between communication process



Socket programming

Two socket types for two transport services:

- **UDP:** unreliable datagram

A

1. Client (ta) from its keyboard and sends the data to the server.
2. The server receives the data and converts characters to uppercase.
3. The server sends the modified data to the client.
4. The client receives the modified data and displays the line on its screen.

~~UDP~~

server

- from r

received out-of-order

Application viewpoint:

- of bytes ("datagrams") between client and server

Client/server socket interaction: UDP

server (running on serverIP)

client

create socket, port= x:
`serverSocket =`

↓
read datagram from
`serverSocket`

↓
write reply to
`serverSocket`
specifying
client address,
port number

create socket:

`clientSocket =`
`ket(AF_INET,SOCK_DGRAM)`

`clientSocket`

↓
read datagram from
`clientSocket`

↓
close
`clientSocket`

P and
gram via

Example app: UDP client

Python UDPClient

include Python's socket
library

→ from socket import *
serverName = 'hostname'
serv

creat
serve

get user keyboard
input

→ message = raw_input('Input lowercase sentence:')
RAM)

Attach server name, port to
message; send into socket

→ clientSocket.sendto(message,(serverName, serverPort))

read reply characters from
socket into string

→ modifiedMessage, serverAddress =
clientSocket.recvfrom(2048)

print out received string
and close socket

→ print modifiedMessage
clientSocket.close()

Example app: UDP server

Python UDPServer

```
from socket import *  
serverPort = 12000
```

create UDP socket → `serverSocket = socket(AF_INET, SOCK_DGRAM)`

bind s
numb

loop forever → `while 1:`

Read from UDP socket into
message, getting client's
address (client IP and port) → `message, clientAddress = serverSocket.recvfrom(2048)`
`modifiedMessage = message.upper()`

send upper case string
back to this client → `serverSocket.sendto(modifiedMessage, clientAddress)`

Socket programming *with* ~~TCP~~

client must contact
server

- ❖ server process must first be running

- ❖ when contacted by client, *server TCP creates new socket* for server process to unicate with that

contac

clients

client contacts server
by:

- ❖ Creating TCP socket, specifying IP address, port number of server process
- ❖ *when client creates socket:* client TCP establishes connection

- source port numbers used to distinguish clients (more in Chap 3)

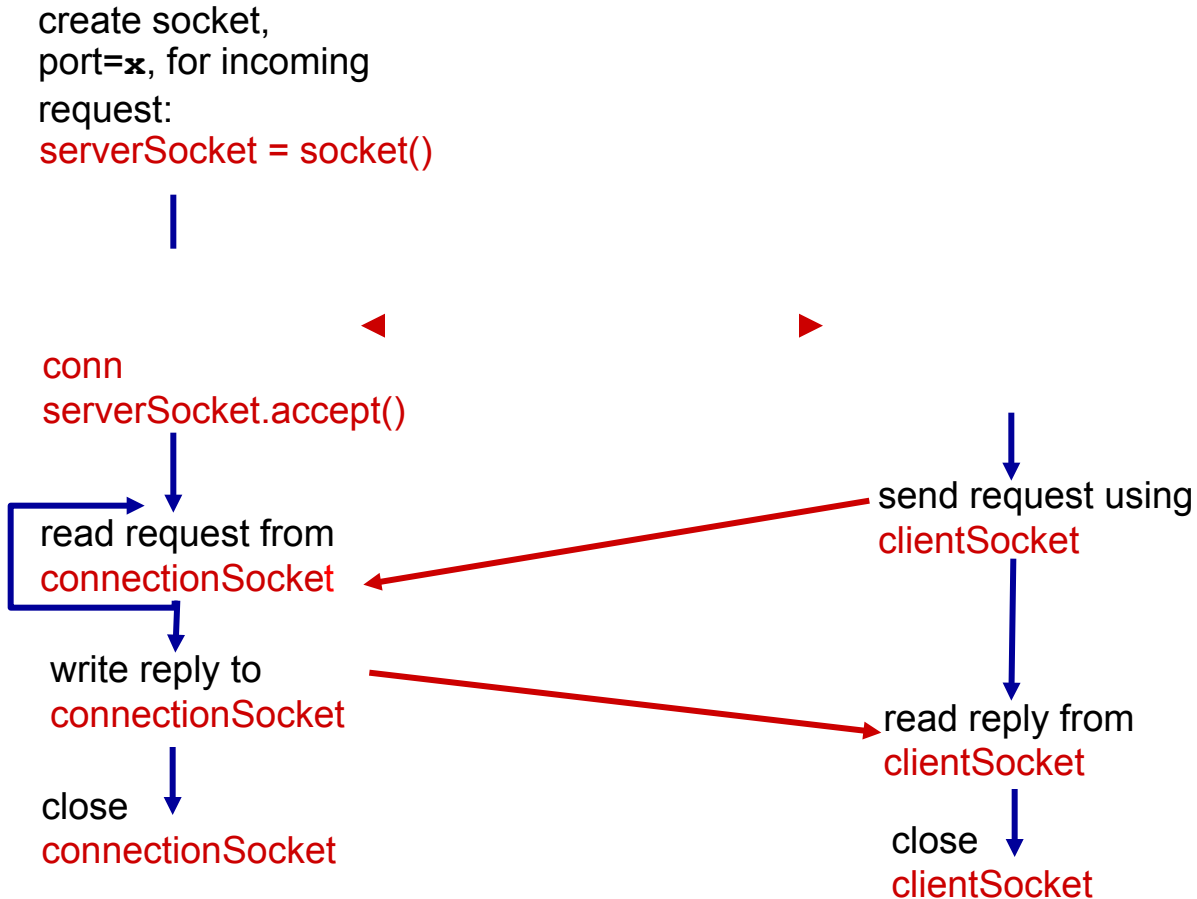
application viewpoint:

TCP provides reliable, in-order byte-stream transfer ("pipe" between client and server)

Client/server socket interaction: TCP

server (running on hostid)

client

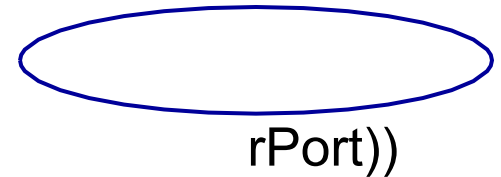


Example app: TCP client

Python TCPClient

```
from socket import *  
serverName = 'servername'  
serv
```

create TCP socket for
server, r



```
sentence = raw_input('Input lowercase sentence:')  
clientSocket.send(sentence)  
modifiedSentence = clientSocket.recv(1024)  
print 'From Server:', modifiedSentence  
clientSocket.close()
```

No need to attach server
name, port

→

Example app: TCP server

Python TCPServer

create TCP welcoming socket	→	serv	INET,SOCK_STREAM)
server incom	→		
loop forever	→	while 1:	
server waits on accept() for incoming requests, new socket created on return	→	connectionSocket, addr = serverSocket.accept()	
read bytes from socket (but not address as in UDP)	→	sentence = connectionSocket.recv(1024) capitalizedSentence = sentence.upper()	
close connection to this client (but <i>not</i> welcoming socket)	→	connectionSocket.send(capitalizedSentence) connectionSocket.close()	

Chapter 2:

~~summary~~

our study of network apps now complete!

- ❖ application architectures
 - client-server
 - P2P
- ❖ Internet transport service model
 - reliability, bandwidth, delay
 - connection-oriented, reliable: TCP
 - unreliable, datagrams: UDP
- ❖ specific protocols:
 - HTTP
 - P2P: BitTorrent, DHT
- ❖ socket programming:
 - TCP, UDP sockets

Chapter 2:

~~summary~~

most importantly: learned about protocols!

❖ typical request/reply message excha

important themes:

ol vs. data msgs

- ser
with data, status
code
- ❖ message formats:
 - headers: fields
giving info about
data
 - data: info being
communicated

- ❖ centralized vs.
decentralized
- ❖ stateless vs. stateful
- ❖ reliable vs. unreliable
msg transfer
- ❖ “complexity at
network edge”

Chapter 1

Additional

